Python_Codes_for_MSL_Recognition

June 10, 2026

1 Python Codes for MSL-Video-Recognition

1.1 augment.py

[1]:
```
from IPython.display import Markdown

file_path = './src/augment.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
augment.py - Keypoint-space data augmentation for MSL Recognition

Correct experimental design for 1-sample-per-class datasets

With 558 videos and ~556 unique classes, a random train/val/test split
produces ZERO class overlap between splits - making val/test accuracy
permanently 0% (the model is asked to classify classes it never trained on).

The correct approach:
  For each of the 558 original videos, generate aug_factor augmented copies.
  Then split BY AUGMENTATION TYPE, not by sample:

    Train  : aug copies 1..aug_factor-1  (all 558 classes, many copies)
    Val    : aug copy   0                (all 558 classes, 1 aug copy each)
    Test   : the original .npy           (all 558 classes, 1 original each)

  Result: 100% class overlap across all three splits.
  The final test evaluates on REAL original keypoints - the strictest test.

Augmentation strategies (10 total)
```

```
 1. GaussianNoise      - coordinate jitter
 2. TimeWarp           - non-linear temporal warp via cubic spline
 3. HorizontalFlip     - mirror x + swap L/R hand blocks
 4. SpeedPerturbation  - resample to random speed factor
 5. FrameDropout       - randomly drop frames then restore length
 6. SpatialRotate      - 2-D rotation in x-y plane
 7. JointMask          - zero-out random joints (occlusion simulation)
 8. ScaleJitter        - global scale perturbation
 9. TemporalShift      - circular time shift
10. CombinedRandom     - random subset of the above
"""

import argparse
import json
import random
from pathlib import Path
from typing import List, Optional

import numpy as np
from scipy.interpolate import CubicSpline
from tqdm import tqdm

#   Node layout

POSE_N           = 33
HAND_N           = 21
LEFT_HAND_START  = POSE_N           # 33
RIGHT_HAND_START = POSE_N + HAND_N  # 54

POSE_MIRROR_PAIRS = [
    (1,4),(2,5),(3,6),(7,8),(9,10),
    (11,12),(13,14),(15,16),(17,18),(19,20),(21,22),
    (23,24),(25,26),(27,28),(29,30),(31,32),
]


#   Individual augmentations

def augment_gaussian_noise(seq: np.ndarray, std: float = 0.008) -> np.ndarray:
    return (seq + np.random.normal(0, std, seq.shape)).astype(np.float32)


def augment_time_warp(seq: np.ndarray, sigma: float = 0.15, knot: int = 4) -> np.ndarray:
    T = seq.shape[0]
    if T < 4:
        return seq
    orig_steps = np.linspace(0, T - 1, num=knot + 2)
    warp_steps = orig_steps + np.random.normal(0, sigma * T, orig_steps.shape)
```

```
    warp_steps[0]  = 0
    warp_steps[-1] = T - 1
    warp_steps     = np.clip(warp_steps, 0, T - 1)
    warp_steps     = np.sort(warp_steps)
    # Enforce strictly increasing (CubicSpline requirement)
    eps = 1e-4
    for i in range(1, len(warp_steps)):
        if warp_steps[i] <= warp_steps[i - 1]:
            warp_steps[i] = warp_steps[i - 1] + eps
    if warp_steps[-1] > T - 1:
        return seq.copy()
    cs         = CubicSpline(warp_steps, orig_steps)
    new_steps  = np.linspace(0, T - 1, T)
    orig_times = np.clip(cs(new_steps), 0, T - 1)
    warped = np.zeros_like(seq)
    for j in range(seq.shape[1]):
        for c in range(seq.shape[2]):
            warped[:, j, c] = np.interp(orig_times, np.arange(T), seq[:, j, c])
    return warped.astype(np.float32)


def augment_horizontal_flip(seq: np.ndarray) -> np.ndarray:
    flipped = seq.copy()
    flipped[:, :, 0] = -flipped[:, :, 0]
    for i, j in POSE_MIRROR_PAIRS:
        flipped[:, [i, j], :] = flipped[:, [j, i], :]
    lh = flipped[:, LEFT_HAND_START:LEFT_HAND_START+HAND_N, :].copy()
    rh = flipped[:, RIGHT_HAND_START:RIGHT_HAND_START+HAND_N, :].copy()
    flipped[:, LEFT_HAND_START:LEFT_HAND_START+HAND_N, :]  = rh
    flipped[:, RIGHT_HAND_START:RIGHT_HAND_START+HAND_N, :] = lh
    return flipped.astype(np.float32)


def augment_speed_perturbation(seq: np.ndarray,
                               min_factor: float = 0.75,
                               max_factor: float = 1.35) -> np.ndarray:
    T      = seq.shape[0]
    factor = np.random.uniform(min_factor, max_factor)
    new_T  = max(4, int(round(T / factor)))
    idx    = np.linspace(0, T - 1, new_T)
    out    = np.zeros((new_T, seq.shape[1], seq.shape[2]), dtype=np.float32)
    for j in range(seq.shape[1]):
        for c in range(seq.shape[2]):
            out[:, j, c] = np.interp(idx, np.arange(T), seq[:, j, c])
    return out


def augment_frame_dropout(seq: np.ndarray, max_drop_ratio: float = 0.10) -> np.ndarray:
```

```
    T    = seq.shape[0]
    drop = max(1, int(T * np.random.uniform(0, max_drop_ratio)))
    keep = sorted(random.sample(range(T), T - drop))
    kept = seq[keep]
    idx  = np.round(np.linspace(0, len(keep) - 1, T)).astype(int)
    return kept[idx].astype(np.float32)


def augment_spatial_rotate(seq: np.ndarray, max_angle_deg: float = 12.0) -> np.ndarray:
    angle = np.radians(np.random.uniform(-max_angle_deg, max_angle_deg))
    c, s  = np.cos(angle), np.sin(angle)
    rot   = np.array([[c, -s], [s, c]], dtype=np.float32)
    out   = seq.copy()
    out[:, :, :2] = np.einsum('ij,tnj->tni', rot, seq[:, :, :2])
    return out.astype(np.float32)


def augment_joint_mask(seq: np.ndarray, max_mask_ratio: float = 0.10) -> np.ndarray:
    N       = seq.shape[1]
    n_mask  = max(1, int(N * np.random.uniform(0, max_mask_ratio)))
    indices = random.sample(range(N), n_mask)
    out     = seq.copy()
    out[:, indices, :] = 0.0
    return out.astype(np.float32)


def augment_scale_jitter(seq: np.ndarray,
                         min_scale: float = 0.85,
                         max_scale: float = 1.15) -> np.ndarray:
    return (seq * np.random.uniform(min_scale, max_scale)).astype(np.float32)


def augment_temporal_shift(seq: np.ndarray, max_shift: int = 5) -> np.ndarray:
    shift = np.random.randint(-max_shift, max_shift + 1)
    return np.roll(seq, shift, axis=0).astype(np.float32)


#    Combined random augmentor

class RandomAugmentor:
    """Apply a random subset of augmentations; each with its own probability."""

    def __init__(self, cfg: dict):
        self.cfg = cfg

    def _p(self, key: str) -> float:
        return self.cfg.get(key, {}).get('prob', 0.0)
```

```
    def __call__(self, seq: np.ndarray) -> np.ndarray:
        aug = seq.copy()

        if random.random() < self._p('gaussian_noise'):
            aug = augment_gaussian_noise(aug, std=self.cfg['gaussian_noise'].get('std', 0.008))

        if random.random() < self._p('time_warp'):
            aug = augment_time_warp(aug,
                sigma=self.cfg['time_warp'].get('sigma', 0.15),
                knot =self.cfg['time_warp'].get('knot',  4))

        if random.random() < self._p('horizontal_flip'):
            aug = augment_horizontal_flip(aug)

        if random.random() < self._p('speed_perturbation'):
            sp  = self.cfg['speed_perturbation']
            aug = augment_speed_perturbation(aug,
                min_factor=sp.get('min_factor', 0.75),
                max_factor=sp.get('max_factor', 1.35))

        if random.random() < self._p('frame_dropout'):
            aug = augment_frame_dropout(aug,
                max_drop_ratio=self.cfg['frame_dropout'].get('max_drop_ratio', 0.10))

        if random.random() < self._p('spatial_rotate'):
            aug = augment_spatial_rotate(aug,
                max_angle_deg=self.cfg['spatial_rotate'].get('max_angle', 12))

        if random.random() < self._p('joint_mask'):
            aug = augment_joint_mask(aug,
                max_mask_ratio=self.cfg['joint_mask'].get('max_mask_ratio', 0.10))

        if random.random() < self._p('scale_jitter'):
            sj  = self.cfg['scale_jitter']
            aug = augment_scale_jitter(aug,
                min_scale=sj.get('min_scale', 0.85),
                max_scale=sj.get('max_scale', 1.15))

        if random.random() < self._p('temporal_shift'):
            aug = augment_temporal_shift(aug,
                max_shift=self.cfg['temporal_shift'].get('max_shift', 5))

        return aug


#   Main augmentation pipeline (correct design for 1-sample-per-class)

def augment_dataset_all_classes(
```

```
    keypoint_dir: str,
    output_dir:   str,
    records:      list,       # ALL 558 matched records
    label2idx:    dict,
    aug_factor:   int  = 20,  # total augmented copies per original
    aug_cfg:      dict = None,
    seed:         int  = 42,
) -> dict:
    """
    Correct augmentation strategy for datasets with 1 sample per class.

    For EACH of the 558 original videos:
      - aug_000  → val split   (1 augmented copy, realistic noise)
      - aug_001 .. aug_{N-1} → train split  (N-1 augmented copies)
      - original .npy        → test split  (real, un-augmented)

    This guarantees 100% class overlap across all three splits.

    Returns:
        dict with keys 'train', 'val', 'test', each a list of record dicts.
        Also saves augmented_manifest.json in output_dir.
    """
    random.seed(seed)
    np.random.seed(seed)

    output_dir = Path(output_dir)
    train_dir  = output_dir / 'train'
    val_dir    = output_dir / 'val'
    train_dir.mkdir(parents=True, exist_ok=True)
    val_dir.mkdir(parents=True, exist_ok=True)

    augmentor = RandomAugmentor(aug_cfg) if aug_cfg else None

    train_records = []
    val_records   = []
    test_records  = []   # points to original keypoint files

    missing = 0
    for rec in tqdm(records, desc="Augmenting all classes", unit="sign"):
        kp_path = rec.get('keypoint_path')
        if not kp_path or not Path(kp_path).exists():
            missing += 1
            continue

        seq   = np.load(kp_path).astype(np.float32)  # (T, 75, 3)
        idx   = rec['idx']
        label = rec['label']
```

```
        #   Test: original keypoint (no augmentation)
        test_records.append({
            **rec,
            'keypoint_path': kp_path,
            'is_augmented':  False,
            'split':         'test',
        })

        #   Val: first augmented copy (aug_000)
        aug0_path = val_dir / f"{idx:04d}_aug000.npy"
        aug0      = augmentor(seq) if augmentor else seq.copy()
        np.save(str(aug0_path), aug0)
        val_records.append({
            **rec,
            'keypoint_path': str(aug0_path),
            'is_augmented':  True,
            'aug_id':        0,
            'split':         'val',
        })

        #   Train: aug_001 .. aug_{aug_factor-1}
        for aug_i in range(1, aug_factor):
            aug_path = train_dir / f"{idx:04d}_aug{aug_i:03d}.npy"
            aug_seq  = augmentor(seq) if augmentor else seq.copy()
            np.save(str(aug_path), aug_seq)
            train_records.append({
                **rec,
                'keypoint_path': str(aug_path),
                'is_augmented':  True,
                'aug_id':        aug_i,
                'split':         'train',
            })

if missing:
    print(f"[WARNING] {missing} records had no keypoint file and were skipped.")

manifest = {
    'train': train_records,
    'val':   val_records,
    'test':  test_records,
}

manifest_path = output_dir / 'augmented_manifest.json'
with open(manifest_path, 'w', encoding='utf-8') as f:
    json.dump(manifest, f, ensure_ascii=False, indent=2)

print(f"\n{'='*60}")
print("  Augmentation Complete (all-class design)")
```

```
    print(f"{'='*60}")
    print(f"  Original videos   : {len(records)}")
    print(f"  Aug factor        : {aug_factor}  (train uses {aug_factor-1}, val uses 1)")
    print(f"  Train samples     : {len(train_records)}  (augmented, all classes)")
    print(f"  Val   samples     : {len(val_records)}   (augmented, all classes)")
    print(f"  Test  samples     : {len(test_records)}   (original,  all classes)")
    print(f"  Class overlap     : 100% across all splits")
    print(f"  Manifest          : {manifest_path}")
    print(f"{'='*60}\n")

    return manifest


#    CLI

def main():
    import yaml
    import sys
    sys.path.insert(0, str(Path(__file__).parent))
    from utils import (parse_annotation_file, build_label_vocabulary,
                       match_videos_to_annotations, get_logger)

    parser = argparse.ArgumentParser(
        description="MSL keypoint augmentation (all-class design for 1-sample-per-class dataset
    )
    parser.add_argument('--config',      required=True)
    parser.add_argument('--aug_factor',  type=int, default=None)
    args = parser.parse_args()

    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    dcfg = cfg['data']
    acfg = cfg['augmentation']
    if args.aug_factor:
        acfg['aug_factor'] = args.aug_factor

    records              = parse_annotation_file(dcfg['annotation_file'])
    label2idx, idx2label = build_label_vocabulary(records)
    records              = match_videos_to_annotations(
        dcfg['video_dir'], records, dcfg['keypoint_dir']
    )

    augment_dataset_all_classes(
        keypoint_dir = dcfg['keypoint_dir'],
        output_dir   = dcfg['augmented_dir'],
        records      = records,
        label2idx    = label2idx,
```

```
        aug_factor   = acfg.get('aug_factor', 20),
        aug_cfg      = acfg,
        seed         = dcfg.get('seed', 42),
    )


if __name__ == '__main__':
    main()
```

1.2 cross_validate.py

[2]:
```
from IPython.display import Markdown

file_path = './src/cross_validate.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
cross_validate.py - K-Fold Cross-Validation for MSL Recognition

Correct experimental design for 1-sample-per-class datasets

The same split-design problem that caused 0% in train/val/test also
affects naive K-Fold: splitting 558 raw records into K folds gives
~446 train / ~112 val with ZERO class overlap → 0% every fold.

Fix: fold over the AUGMENTED training pool from the manifest.
  - The manifest['train'] has 10,602 records (558 classes × 19 aug copies)
  - Every class has 19 copies → every class appears in BOTH train and val
    within every fold
  - Final test is always manifest['test'] (558 originals, strictest eval)

K-Fold structure (e.g. K=5, 10,602 train samples):
  Each fold:
    train = 4/5 × 10,602  8,481 augmented samples  (all 558 classes)
    val   = 1/5 × 10,602  2,121 augmented samples  (all 558 classes)
    test  = 558 originals (same across all folds, for final comparison)

Usage

  python src/cross_validate.py \
```

```
      --config config/config.yaml \
      --model  bilstm \
      --exp    cv_bilstm \
      --folds  5
"""

import argparse
import json
import random
import sys
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
import yaml
from sklearn.model_selection import KFold
from torch.amp import GradScaler
from torch.utils.data import DataLoader

sys.path.insert(0, str(Path(__file__).parent))
from dataset import MSLDataset, MSLDatasetGCN
from models  import build_model
from utils   import (
    get_logger, set_seed, get_device,
    load_label_map, compute_class_weights, save_checkpoint,
)
from train   import train_one_epoch, validate, FocalLoss


#   Single fold

def run_fold(
    fold_idx:   int,
    train_recs: list,
    val_recs:   list,
    test_recs:  list,
    label2idx:  dict,
    cfg:        dict,
    model_type: str,
    exp_dir:    Path,
    device:     torch.device,
    logger,
) -> dict:
    """Train and evaluate one fold. Returns result dict."""

    tcfg     = cfg['training']
```

```
dcfg     = cfg['data']
fold_dir = exp_dir / f'fold_{fold_idx:02d}'
fold_dir.mkdir(parents=True, exist_ok=True)

set_seed(tcfg['seed'] + fold_idx)

DatasetClass = MSLDatasetGCN if model_type == 'stgcn' else MSLDataset
flatten      = (model_type != 'stgcn')
max_T        = dcfg['max_seq_len']

# No on-the-fly augmentation - data is already augmented from manifest
train_ds = DatasetClass(train_recs, label2idx, max_seq_len=max_T,
                        flatten=flatten, augmentor=None)
val_ds   = DatasetClass(val_recs,   label2idx, max_seq_len=max_T,
                        flatten=flatten, augmentor=None)
test_ds  = DatasetClass(test_recs,  label2idx, max_seq_len=max_T,
                        flatten=flatten, augmentor=None)

train_loader = DataLoader(train_ds, batch_size=tcfg['batch_size'], shuffle=True,
                          drop_last=True,  num_workers=tcfg['num_workers'], pin_memory=True
val_loader   = DataLoader(val_ds,   batch_size=tcfg['batch_size'], shuffle=False,
                          drop_last=False, num_workers=tcfg['num_workers'], pin_memory=True
test_loader  = DataLoader(test_ds,  batch_size=tcfg['batch_size'], shuffle=False,
                          drop_last=False, num_workers=tcfg['num_workers'], pin_memory=True

logger.info(
    f"Fold {fold_idx} | "
    f"train={len(train_ds)}, val={len(val_ds)}, test={len(test_ds)}"
)

num_classes   = len(label2idx)
model         = build_model(model_type, cfg, num_classes).to(device)
class_weights = compute_class_weights(train_recs, label2idx, device)

if tcfg.get('loss', 'cross_entropy') == 'focal':
    criterion = FocalLoss(
        gamma=tcfg.get('focal_gamma', 2.0),
        weight=class_weights,
        label_smoothing=tcfg.get('label_smoothing', 0.0),
    )
else:
    criterion = nn.CrossEntropyLoss(
        weight=class_weights,
        label_smoothing=tcfg.get('label_smoothing', 0.0),
    )

optimizer = torch.optim.AdamW(
    model.parameters(), lr=tcfg['learning_rate'], weight_decay=tcfg['weight_decay']
```

```
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=tcfg['num_epochs'], eta_min=tcfg.get('cosine_eta_min', 1e-6)
)
scaler = GradScaler('cuda', enabled=tcfg['use_amp'])

# Start at -1 so epoch 1 always writes a checkpoint even at 0% accuracy
best_val_top1  = -1.0
patience_count = 0
patience       = tcfg['patience']
history        = []

for epoch in range(1, tcfg['num_epochs'] + 1):
    t0            = time.time()
    train_metrics = train_one_epoch(model, train_loader, optimizer, criterion,
                                    scaler, device, cfg, logger)
    val_metrics   = validate(model, val_loader, criterion, device, cfg)
    scheduler.step()

    history.append({
        'epoch': epoch,
        **train_metrics,
        **{f'val_{k}': v for k, v in val_metrics.items()},
    })

    is_best = val_metrics['top1'] > best_val_top1
    if is_best:
        best_val_top1  = val_metrics['top1']
        patience_count = 0
        save_checkpoint(
            {
                'epoch': epoch, 'model_type': model_type,
                'state_dict': model.state_dict(),
                'val_top1': best_val_top1,
                'num_classes': num_classes, 'config': cfg,
            },
            str(fold_dir / 'best.pth'),
            is_best=False,
        )
    else:
        patience_count += 1

    if epoch % 10 == 0 or is_best:
        logger.info(
            f"  Fold {fold_idx} | Epoch {epoch:3d} | "
            f"train={train_metrics['top1']*100:.1f}% | "
            f"val={val_metrics['top1']*100:.2f}%"
            f"{'  ' if is_best else ''} | {time.time()-t0:.0f}s"
```

```
            )

        if patience_count >= patience:
            logger.info(f"  Fold {fold_idx}: early stopping at epoch {epoch}")
            break

    with open(fold_dir / 'history.json', 'w') as f:
        json.dump(history, f, indent=2)

    # Evaluate best checkpoint on test set
    test_ckpt = torch.load(str(fold_dir / 'best.pth'), map_location=device)
    model.load_state_dict(test_ckpt['state_dict'])
    test_metrics = validate(model, test_loader, criterion, device, cfg)
    logger.info(
        f"  Fold {fold_idx} FINAL | "
        f"best_val={best_val_top1*100:.2f}% | "
        f"test={test_metrics['top1']*100:.2f}%"
    )

    return {
        'fold':          fold_idx,
        'best_val_top1': best_val_top1,
        'test_top1':     test_metrics['top1'],
        'best_epoch':    max(history, key=lambda h: h.get('val_top1', 0))['epoch'],
        'checkpoint':    str(fold_dir / 'best.pth'),
    }


#   Main

def main():
    parser = argparse.ArgumentParser(
        description='K-Fold Cross-Validation for MSL Recognition '
                    '(folds over augmented manifest, all-class design)'
    )
    parser.add_argument('--config',     default='config/config.yaml')
    parser.add_argument('--model',      default='bilstm',
                        choices=['bilstm', 'transformer', 'stgcn'])
    parser.add_argument('--exp',        default='cv_exp01')
    parser.add_argument('--folds',      type=int, default=None)
    parser.add_argument('--fold_start', type=int, default=0,
                        help='Resume from this fold index')
    args = parser.parse_args()

    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    dcfg    = cfg['data']
```

```
set_seed(cfg['training']['seed'])

exp_dir = Path('results') / args.exp
exp_dir.mkdir(parents=True, exist_ok=True)

logger = get_logger('cross_validate', log_file=str(exp_dir / 'cv.log'))
device = get_device()
n_folds = args.folds or dcfg.get('n_folds', 5)

#   Load label map
label2idx, idx2label = load_label_map(dcfg['label_map_file'])

#   Load augmented manifest
aug_manifest_path = Path(dcfg['augmented_dir']) / 'augmented_manifest.json'
if not aug_manifest_path.exists():
    logger.error(
        f"Augmented manifest not found: {aug_manifest_path}\n"
        "Run: bash scripts/03_augment_data.sh"
    )
    sys.exit(1)

with open(aug_manifest_path, encoding='utf-8') as f:
    manifest = json.load(f)

if not (isinstance(manifest, dict) and 'train' in manifest):
    logger.error(
        "Old manifest format detected. "
        "Re-run: bash scripts/03_augment_data.sh --force"
    )
    sys.exit(1)

all_train_recs = manifest['train']   # 10,602 aug samples, all 558 classes
test_recs      = manifest['test']    # 558 originals - used as test in every fold

logger.info("=" * 60)
logger.info(f"Cross-Validation | model={args.model} | folds={n_folds}")
logger.info("Folding over augmented training pool (all-class design)")
logger.info(f"  Aug train pool : {len(all_train_recs)} samples")
logger.info(f"  Test set       : {len(test_recs)} originals (fixed, all folds)")
logger.info("=" * 60)

#   Build K-Fold indices over the augmented train pool
# Every class has 19 aug copies in the pool → every class appears in
# both train and val within each fold → meaningful accuracy signal.
indices = np.arange(len(all_train_recs))
kf      = KFold(n_splits=n_folds, shuffle=True,
                random_state=cfg['training']['seed'])
```

```
fold_splits = [
    {'fold': i, 'train': tr.tolist(), 'val': va.tolist()}
    for i, (tr, va) in enumerate(kf.split(indices))
]

# Save the fold splits for reproducibility
with open(exp_dir / 'cv_fold_splits.json', 'w') as f:
    json.dump(fold_splits, f, indent=2)

fold_results = []

for fold_data in fold_splits:
    fold_idx = fold_data['fold']
    if fold_idx < args.fold_start:
        logger.info(f"Skipping fold {fold_idx}")
        continue

    train_recs = [all_train_recs[i] for i in fold_data['train']]
    val_recs   = [all_train_recs[i] for i in fold_data['val']]

    result = run_fold(
        fold_idx   = fold_idx,
        train_recs = train_recs,
        val_recs   = val_recs,
        test_recs  = test_recs,
        label2idx  = label2idx,
        cfg        = cfg,
        model_type = args.model,
        exp_dir    = exp_dir,
        device     = device,
        logger     = logger,
    )
    fold_results.append(result)
    logger.info(
        f"Fold {fold_idx} done | "
        f"val={result['best_val_top1']*100:.2f}% | "
        f"test={result['test_top1']*100:.2f}%"
    )

#   Aggregate
val_scores  = [r['best_val_top1'] for r in fold_results]
test_scores = [r['test_top1']     for r in fold_results]
best_fold   = max(fold_results, key=lambda r: r['best_val_top1'])

summary = {
    'model':             args.model,
    'n_folds':           len(fold_results),
    'fold_val_scores':   [round(s * 100, 2) for s in val_scores],
```

```
        'fold_test_scores':  [round(s * 100, 2) for s in test_scores],
        'mean_val_top1':     round(float(np.mean(val_scores))  * 100, 2),
        'std_val_top1':      round(float(np.std(val_scores))   * 100, 2),
        'mean_test_top1':    round(float(np.mean(test_scores)) * 100, 2),
        'std_test_top1':     round(float(np.std(test_scores))  * 100, 2),
        'best_fold':         best_fold['fold'],
        'best_top1':         round(best_fold['best_val_top1']  * 100, 2),
        # Keep these keys for plot_results.py compatibility
        'fold_scores':       [round(s * 100, 2) for s in val_scores],
        'mean_top1':         round(float(np.mean(val_scores))  * 100, 2),
        'std_top1':          round(float(np.std(val_scores))   * 100, 2),
        'best_checkpoint':   best_fold['checkpoint'],
        'fold_details':      fold_results,
    }

    with open(exp_dir / 'cv_summary.json', 'w') as f:
        json.dump(summary, f, indent=2)

    print(f"\n{'='*65}")
    print(f"  Cross-Validation Summary  [{args.model}]")
    print(f"{'='*65}")
    print(f"  {'Fold':<8} {'Val Top-1':>10} {'Test Top-1':>12}")
    print(f"  {' '*34}")
    for r in fold_results:
        print(f"  Fold {r['fold']}   {r['best_val_top1']*100:>8.2f}%   "
              f"{r['test_top1']*100:>9.2f}%")
    print(f"  {' '*34}")
    print(f"  {'Mean±Std':<8} "
          f"{summary['mean_val_top1']:>7.2f}±{summary['std_val_top1']:.2f}%   "
          f"{summary['mean_test_top1']:>7.2f}±{summary['std_test_top1']:.2f}%")
    print(f"  Best fold : {best_fold['fold']}  (val {summary['best_top1']:.2f}%)")
    print(f"{'='*65}\n")


if __name__ == '__main__':
    main()
```

1.3 dataset.py

[3]:
```
from IPython.display import Markdown

file_path = './src/dataset.py'

with open(file_path, 'r') as f:
    code = f.read()
```

```
# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
dataset.py - PyTorch Dataset for Myanmar Sign Language Recognition

Supports:
  - Loading pre-extracted keypoint sequences (.npy)
  - On-the-fly augmentation (for training)
  - Padding / truncation to fixed length
  - Returning variable-length sequences with masks
"""

from pathlib import Path
from typing import Dict, List, Optional, Tuple

import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

from augment import RandomAugmentor


#    Dataset

class MSLDataset(Dataset):
    """
    Myanmar Sign Language keypoint dataset.

    Each item is a dict:
      {
        'keypoints': FloatTensor (T, 75, 3)  - or flattened (T, 225),
        'label':     LongTensor  ()
        'length':    int  - original (unpadded) sequence length,
        'mask':      BoolTensor  (max_seq_len,)  - True where padded,
        'idx':       int  - annotation index,
      }
    """

    def __init__(
        self,
        records:      List[Dict],
        label2idx:    Dict[str, int],
        max_seq_len:  int  = 200,
        flatten:      bool = True,     # (T,75,3) → (T,225)
        augmentor:    Optional[RandomAugmentor] = None,
```

```
    min_seq_len:  int  = 5,
):
    """
    Args:
        records:     list of dicts with 'keypoint_path' and 'label' keys.
        label2idx:   mapping from label string to class index.
        max_seq_len: sequences longer than this are centre-cropped;
                     shorter ones are zero-padded.
        flatten:     if True, return (T, 225) instead of (T, 75, 3).
        augmentor:   RandomAugmentor instance for on-the-fly augmentation.
        min_seq_len: samples with fewer than this many frames are skipped.
    """
    self.label2idx   = label2idx
    self.max_seq_len = max_seq_len
    self.flatten     = flatten
    self.augmentor   = augmentor
    self.min_seq_len = min_seq_len

    # Filter out records with missing keypoint files
    self.samples = []
    missing = 0
    for rec in records:
        kp = rec.get('keypoint_path')
        if kp and Path(kp).exists():
            self.samples.append(rec)
        else:
            missing += 1

    if missing:
        print(f"[MSLDataset] Warning: {missing} samples have no keypoint file.")

def __len__(self) -> int:
    return len(self.samples)

def __getitem__(self, idx: int) -> Dict:
    rec    = self.samples[idx]
    seq    = np.load(rec['keypoint_path']).astype(np.float32)  # (T, 75, 3)
    label  = self.label2idx[rec['label']]

    # On-the-fly augmentation
    if self.augmentor is not None:
        seq = self.augmentor(seq)

    # Ensure minimum length
    T = seq.shape[0]
    if T < self.min_seq_len:
        # Repeat sequence until long enough
        reps = int(np.ceil(self.min_seq_len / T))
```

```
            seq  = np.tile(seq, (reps, 1, 1))[:self.min_seq_len]
            T    = self.min_seq_len

        # Centre-crop if too long
        if T > self.max_seq_len:
            start = (T - self.max_seq_len) // 2
            seq   = seq[start: start + self.max_seq_len]
            T     = self.max_seq_len

        length = T  # true length before padding

        # Zero-pad to max_seq_len
        pad_len = self.max_seq_len - T
        if pad_len > 0:
            pad_shape = (pad_len, seq.shape[1], seq.shape[2])
            seq = np.concatenate([seq, np.zeros(pad_shape, dtype=np.float32)], axis=0)

        # Padding mask: True where padded (for Transformer key_padding_mask)
        mask = np.zeros(self.max_seq_len, dtype=bool)
        mask[length:] = True

        if self.flatten:
            seq = seq.reshape(self.max_seq_len, -1)  # (T, 225)

        return {
            'keypoints': torch.from_numpy(seq),
            'label':     torch.tensor(label, dtype=torch.long),
            'length':    torch.tensor(length, dtype=torch.long),
            'mask':      torch.from_numpy(mask),
            'idx':       rec['idx'],
        }


class MSLDatasetGCN(MSLDataset):
    """
    Variant that returns keypoints in ST-GCN format: (C, T, V)
      C = coordinate channels (3)
      T = time steps
      V = nodes (75)
    """

    def __getitem__(self, idx: int) -> Dict:
        item = super().__getitem__(idx)
        # Reshape from (T, 225) → (T, 75, 3) → (3, T, 75)
        kp = item['keypoints'].reshape(self.max_seq_len, 75, 3)
        kp = kp.permute(2, 0, 1)              # (3, T, 75)
        item['keypoints'] = kp
        return item
```

```python
#   DataLoader factory

def build_dataloaders(
    train_records: List[Dict],
    val_records:   List[Dict],
    test_records:  List[Dict],
    label2idx:     Dict[str, int],
    cfg:           dict,
    augmentor:     Optional[RandomAugmentor] = None,
    model_type:    str = 'bilstm',            # 'bilstm' | 'transformer' | 'stgcn'
) -> Tuple[DataLoader, DataLoader, DataLoader]:
    """
    Build train / val / test DataLoaders.

    For training:  uses augmentor if provided.
    For val/test:  no augmentation, deterministic order.
    """
    dcfg   = cfg['data']
    tcfg   = cfg['training']
    max_T  = dcfg['max_seq_len']

    DatasetClass = MSLDatasetGCN if model_type == 'stgcn' else MSLDataset
    flatten      = (model_type != 'stgcn')

    train_ds = DatasetClass(
        train_records, label2idx,
        max_seq_len = max_T,
        flatten     = flatten,
        augmentor   = augmentor,
    )
    val_ds = DatasetClass(
        val_records, label2idx,
        max_seq_len = max_T,
        flatten     = flatten,
        augmentor   = None,
    )
    test_ds = DatasetClass(
        test_records, label2idx,
        max_seq_len = max_T,
        flatten     = flatten,
        augmentor   = None,
    )

    loader_kwargs = dict(
        batch_size  = tcfg['batch_size'],
        num_workers = tcfg.get('num_workers', 4),
```

```
        pin_memory  = tcfg.get('pin_memory', True),
    )

    train_loader = DataLoader(train_ds, shuffle=True,  drop_last=True,  **loader_kwargs)
    val_loader   = DataLoader(val_ds,   shuffle=False, drop_last=False, **loader_kwargs)
    test_loader  = DataLoader(test_ds,  shuffle=False, drop_last=False, **loader_kwargs)

    print(f"\n[DataLoaders]")
    print(f"  Train: {len(train_ds)} samples, {len(train_loader)} batches")
    print(f"  Val:   {len(val_ds)}   samples, {len(val_loader)} batches")
    print(f"  Test:  {len(test_ds)}  samples, {len(test_loader)} batches\n")

    return train_loader, val_loader, test_loader
```

1.4 evaluate.py

[4]:
```
from IPython.display import Markdown

file_path = './src/evaluate.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
evaluate.py - Comprehensive evaluation for MSL Recognition models

Computes

  Top-1, Top-5 Accuracy
  Precision, Recall, F1 (macro / weighted)
  Per-class metrics
  Confusion matrix (saved as PNG + JSON)
  Prediction CSV with ground-truth vs predicted labels

Usage

  python src/evaluate.py \
      --checkpoint results/exp01_bilstm/checkpoints/best.pth \
      --config     config/config.yaml \
      --split      test
"""

# === FIX: Handle Jupyter's matplotlib backend conflict ===
```

```
import os
_mpl_backend = os.environ.get('MPLBACKEND')
if _mpl_backend and _mpl_backend not in [
    'gtk3agg', 'gtk3cairo', 'gtk4agg', 'gtk4cairo', 'macosx',
    'nbagg', 'notebook', 'qtagg', 'qtcairo', 'qt5agg', 'qt5cairo',
    'tkagg', 'tkcairo', 'webagg', 'wx', 'wxagg', 'wxcairo',
    'agg', 'cairo', 'pdf', 'pgf', 'ps', 'svg', 'template'
]:
    os.environ['MPLBACKEND'] = 'agg'
# === END FIX ===

import argparse
import json
import sys
from pathlib import Path

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import torch
import yaml
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    precision_recall_fscore_support,
    top_k_accuracy_score,
)
from torch.amp import autocast

sys.path.insert(0, str(Path(__file__).parent))
from dataset   import MSLDataset, MSLDatasetGCN
from models    import build_model
from utils     import (
    get_logger, set_seed, get_device,
    parse_annotation_file, build_label_vocabulary,
    match_videos_to_annotations, load_splits, load_label_map,
)

torch.backends.cudnn.deterministic = True


#   Prediction collection

@torch.no_grad()
def collect_predictions(model, loader, device, model_type: str, use_amp: bool):
```

```
    """Run model over loader; return all logits, labels, indices."""
    model.eval()
    all_logits = []
    all_labels = []
    all_idxs   = []

    for batch in loader:
        kp     = batch['keypoints'].to(device, non_blocking=True)
        labels = batch['label'].to(device, non_blocking=True)
        mask   = batch['mask'].to(device, non_blocking=True)
        lengths= batch['length'].to(device, non_blocking=True)

        with autocast('cuda', enabled=use_amp):
            if model_type == 'bilstm':
                logits = model(kp, lengths=lengths, mask=mask)
            elif model_type == 'transformer':
                logits = model(kp, mask=mask)
            else:  # stgcn
                logits = model(kp)

        all_logits.append(logits.cpu())
        all_labels.append(labels.cpu())
        all_idxs.extend(batch['idx'] if isinstance(batch['idx'], list) else batch['idx'].tolist

    return (
        torch.cat(all_logits, dim=0),
        torch.cat(all_labels, dim=0),
        all_idxs,
    )


#   Metrics

def compute_all_metrics(
    logits:    torch.Tensor,
    labels:    torch.Tensor,
    idx2label: dict,
    output_dir: str,
    split_name: str = 'test',
    plot_cm:   bool = True,
    logger = None,
):
    """Compute and save comprehensive metrics."""
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    y_true  = labels.numpy()
    y_prob  = torch.softmax(logits, dim=1).numpy()
```

```
y_pred  = logits.argmax(dim=1).numpy()
n_cls   = logits.shape[1]

# Top-k accuracy
top1 = float((y_pred == y_true).mean())

# top-5: only meaningful when n_cls >= 5 and each class has  1 sample in split
try:
    present_labels = sorted(np.unique(y_true).tolist())
    k5 = min(5, len(present_labels))
    top5 = float(top_k_accuracy_score(y_true, y_prob, k=k5, labels=list(range(n_cls))))
except Exception:
    # Fallback: manual top-5
    top5_preds = np.argsort(y_prob, axis=1)[:, -5:]
    top5 = float(np.mean([y_true[i] in top5_preds[i] for i in range(len(y_true))]))

# Macro and weighted precision/recall/F1
prec_macro, rec_macro, f1_macro, _ = precision_recall_fscore_support(
    y_true, y_pred, average='macro', zero_division=0
)
prec_w, rec_w, f1_w, _ = precision_recall_fscore_support(
    y_true, y_pred, average='weighted', zero_division=0
)

summary = {
    'split':          split_name,
    'num_samples':    int(len(y_true)),
    'num_classes':    n_cls,
    'top1_accuracy':  float(top1),
    'top5_accuracy':  float(top5),
    'precision_macro':   float(prec_macro),
    'recall_macro':      float(rec_macro),
    'f1_macro':          float(f1_macro),
    'precision_weighted': float(prec_w),
    'recall_weighted':    float(rec_w),
    'f1_weighted':        float(f1_w),
}

# Print summary
bar = ' ' * 55
print(f"\n{bar}")
print(f"  Evaluation Results [{split_name.upper()}]")
print(f"{bar}")
print(f"  Samples       : {summary['num_samples']}")
print(f"  Classes       : {summary['num_classes']}")
print(f"  Top-1 Accuracy: {top1*100:.2f}%")
print(f"  Top-5 Accuracy: {top5*100:.2f}%")
print(f"  Precision (M) : {prec_macro*100:.2f}%")
```

```
print(f"  Recall    (M) : {rec_macro*100:.2f}%")
print(f"  F1        (M) : {f1_macro*100:.2f}%")
print(f"{bar}\n")

# Save summary JSON
with open(output_dir / f'metrics_{split_name}.json', 'w', encoding='utf-8') as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)

# Per-class report - only include classes actually present in this split
present_labels = sorted(np.unique(y_true).tolist())
present_names  = [idx2label.get(i, str(i)) for i in present_labels]
report = classification_report(
    y_true, y_pred,
    labels        = present_labels,
    target_names  = present_names,
    zero_division = 0,
    output_dict   = True,
)
report_df = pd.DataFrame(report).T
report_df.to_csv(output_dir / f'per_class_{split_name}.csv', encoding='utf-8-sig')

# Pretty-print top / bottom classes
per_cls = {k: v for k, v in report.items() if k not in ['accuracy', 'macro avg', 'weighted
sorted_f1 = sorted(per_cls.items(), key=lambda x: x[1]['f1-score'], reverse=True)
print("Top-10 classes by F1:")
for name, m in sorted_f1[:10]:
    print(f"  {name[:35]:<35}  F1={m['f1-score']:.3f}  "
          f"Prec={m['precision']:.3f}  Rec={m['recall']:.3f}  "
          f"Support={int(m['support'])}")
print("\nBottom-10 classes by F1:")
for name, m in sorted_f1[-10:]:
    print(f"  {name[:35]:<35}  F1={m['f1-score']:.3f}  "
          f"Prec={m['precision']:.3f}  Rec={m['recall']:.3f}  "
          f"Support={int(m['support'])}")

# Confusion matrix
if plot_cm:
    # Only use classes that are actually present in this split
    present_labels = sorted(np.unique(np.concatenate([y_true, y_pred])).tolist())
    cm      = confusion_matrix(y_true, y_pred, labels=present_labels)
    cm_norm = cm.astype(float) / (cm.sum(axis=1, keepdims=True) + 1e-8)

    np.save(output_dir / f'confusion_matrix_{split_name}.npy', cm)

    n_present = len(present_labels)

    # For large class counts (>60): plot a compact pixel-map without labels.
    # For small counts (<=60): full annotated heatmap.
```

```
        if n_present > 60:
            fig, ax = plt.subplots(figsize=(14, 12))
            im = ax.imshow(cm_norm, aspect='auto', cmap='Blues', vmin=0, vmax=1,
                           interpolation='nearest')
            plt.colorbar(im, ax=ax, fraction=0.03)
            ax.set_xlabel(f'Predicted class index (0-{n_present-1})', fontsize=11)
            ax.set_ylabel(f'True class index (0-{n_present-1})',      fontsize=11)
            ax.set_title(
                f'Confusion Matrix [{split_name}]  '
                f'({n_present} classes present, row-normalised)\n'
                f'Diagonal = correct predictions', fontsize=12
            )
            # Mark diagonal clearly
            diag_acc = np.diag(cm_norm).mean()
            ax.set_title(
                f'Confusion Matrix [{split_name}]  ({n_present}/{n_cls} classes)\n'
                f'Mean diagonal (accuracy): {diag_acc*100:.1f}%', fontsize=12
            )
        else:
            tick_labels = [idx2label.get(i, str(i))[:18] for i in present_labels]
            fig_size    = max(10, n_present // 2)
            fig, ax     = plt.subplots(figsize=(fig_size, fig_size))
            sns.heatmap(
                cm_norm, ax=ax,
                xticklabels=tick_labels, yticklabels=tick_labels,
                cmap='Blues', vmin=0, vmax=1,
                annot=True, fmt='.2f', linewidths=0.4,
                annot_kws={'size': max(5, 9 - n_present // 8)},
            )
            ax.set_xlabel('Predicted', fontsize=12)
            ax.set_ylabel('True',      fontsize=12)
            plt.xticks(rotation=90, fontsize=max(5, 8 - n_present // 10))
            plt.yticks(rotation=0,  fontsize=max(5, 8 - n_present // 10))
            ax.set_title(f'Confusion Matrix [{split_name}]  (row-normalised)', fontsize=13)

        plt.tight_layout()
        plt.savefig(output_dir / f'confusion_matrix_{split_name}.png', dpi=150)
        plt.close()
        print(f"Confusion matrix saved → {output_dir / f'confusion_matrix_{split_name}.png'}")

    return summary


#    Prediction CSV

def save_predictions(
    logits:    torch.Tensor,
    labels:    torch.Tensor,
```

```
    idxs:      list,
    records:   list,
    idx2label: dict,
    output_path: str,
):
    """Save per-sample predictions as CSV."""
    y_true = labels.numpy()
    y_pred = logits.argmax(dim=1).numpy()
    probs  = torch.softmax(logits, dim=1).numpy()

    rows = []
    for i, (true, pred, idx) in enumerate(zip(y_true, y_pred, idxs)):
        rec  = records[idx] if idx < len(records) else {}
        rows.append({
            'sample_idx':    int(idx),
            'true_label':    idx2label.get(int(true), str(true)),
            'pred_label':    idx2label.get(int(pred), str(pred)),
            'correct':       int(true) == int(pred),
            'true_idx':      int(true),
            'pred_idx':      int(pred),
            'confidence':    float(probs[i, int(pred)]),
            'video_path':    rec.get('video_path', ''),
            'msl_gloss':     rec.get('msl_gloss', ''),
        })

    df = pd.DataFrame(rows)
    df.to_csv(output_path, index=False, encoding='utf-8-sig')
    print(f"Predictions saved → {output_path}  "
          f"({df['correct'].sum()}/{len(df)} correct)")
    return df


#   Main

def main():
    parser = argparse.ArgumentParser(description='Evaluate MSL Recognition Model')
    parser.add_argument('--checkpoint', required=True, help='Path to .pth checkpoint')
    parser.add_argument('--config',     default='config/config.yaml')
    parser.add_argument('--split',      default='test', choices=['train', 'val', 'test'])
    parser.add_argument('--output_dir', default=None,
                        help='Where to save results (default: checkpoint directory)')
    parser.add_argument('--no_cm',      action='store_true', help='Skip confusion matrix')
    args = parser.parse_args()

    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    dcfg = cfg['data']
```

```
set_seed(42)
device = get_device()

logger = get_logger('evaluate')

# Load checkpoint
ckpt = torch.load(args.checkpoint, map_location=device)
model_type = ckpt.get('model_type', 'bilstm')
logger.info(f"Loaded checkpoint: {args.checkpoint}  Model: {model_type}")

# Data - load from all-class augmented manifest
records              = parse_annotation_file(dcfg['annotation_file'])
label2idx, idx2label = load_label_map(dcfg['label_map_file'])
records              = match_videos_to_annotations(
    dcfg['video_dir'], records, dcfg['keypoint_dir']
)

aug_manifest_path = Path(dcfg['augmented_dir']) / 'augmented_manifest.json'
with open(aug_manifest_path, encoding='utf-8') as f:
    manifest = json.load(f)

if isinstance(manifest, dict) and args.split in manifest:
    subset = manifest[args.split]
else:
    # Fallback: old flat format - only test split makes sense here
    logger.warning("Old manifest format - falling back to splits.json for this split")
    splits = load_splits(dcfg['split_file'])
    subset = [records[i] for i in splits[args.split]]

logger.info(f"Evaluating {len(subset)} samples from '{args.split}' split")

DatasetClass = MSLDatasetGCN if model_type == 'stgcn' else MSLDataset
dataset = DatasetClass(
    subset, label2idx,
    max_seq_len = dcfg['max_seq_len'],
    flatten     = (model_type != 'stgcn'),
    augmentor   = None,
)
loader = torch.utils.data.DataLoader(
    dataset,
    batch_size  = cfg['training']['batch_size'],
    shuffle     = False,
    num_workers = cfg['training']['num_workers'],
    pin_memory  = True,
)

# Model
num_classes = ckpt.get('num_classes', len(label2idx))
```

```
    model = build_model(model_type, cfg, num_classes).to(device)
    model.load_state_dict(ckpt['state_dict'])
    logger.info(f"Model loaded. Evaluating {len(subset)} samples…")

    # Predict
    logits, labels, idxs = collect_predictions(
        model, loader, device, model_type,
        use_amp = cfg['training']['use_amp']
    )

    # Output dir
    out_dir = args.output_dir or str(Path(args.checkpoint).parent.parent / 'evaluation')

    # Metrics
    summary = compute_all_metrics(
        logits, labels, idx2label,
        output_dir = out_dir,
        split_name = args.split,
        plot_cm    = not args.no_cm,
        logger     = logger,
    )

    # Prediction CSV
    save_predictions(
        logits, labels, idxs, records, idx2label,
        output_path = str(Path(out_dir) / f'predictions_{args.split}.csv'),
    )

    print(f"\nAll evaluation outputs saved to: {out_dir}")


if __name__ == '__main__':
    main()
```

1.5 extract_keypoints.py

[5]:
```
from IPython.display import Markdown

file_path = './src/extract_keypoints.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-
"""
extract_keypoints.py - MediaPipe keypoint extraction for MSL videos
                       (Pose + Hands approach, compatible with mediapipe >= 0.10.13)

Why no Holistic?
  MediaPipe deprecated solutions.holistic in 0.10.1 and removed it entirely
  from builds targeting Python 3.12 (earliest available: 0.10.13).
  This version uses separate Pose and Hands detectors which are still
  supported and produce the identical output tensor shape.

Features extracted per frame:
  - 33 Pose  landmarks x 3 (x, y, z) = 99  floats
  - 21 Left  hand      x 3 (x, y, z) = 63  floats
  - 21 Right hand      x 3 (x, y, z) = 63  floats
  Total : 75 nodes x 3 coords = 225 floats per frame

Output: .npy files shape (T, 75, 3)
        Companion _vis.npy shape (T, 33) - pose visibility scores
"""

# === FIX: Handle Jupyter's matplotlib backend conflict ===
import os
_mpl_backend = os.environ.get('MPLBACKEND')
if _mpl_backend and _mpl_backend not in [
    'gtk3agg', 'gtk3cairo', 'gtk4agg', 'gtk4cairo', 'macosx',
    'nbagg', 'notebook', 'qtagg', 'qtcairo', 'qt5agg', 'qt5cairo',
    'tkagg', 'tkcairo', 'webagg', 'wx', 'wxagg', 'wxcairo',
    'agg', 'cairo', 'pdf', 'pgf', 'ps', 'svg', 'template'
]:
    os.environ['MPLBACKEND'] = 'agg'
# === END FIX ===


import argparse
import json
import logging
from pathlib import Path

import cv2
import numpy as np
from tqdm import tqdm


# === MediaPipe import: Pose + Hands (mediapipe >= 0.10.13) ====================

def _import_mp_modules():
    """Try all known import paths; return (pose_mod, hands_mod, drawing_mod)."""
```

```
    import importlib
    prefixes = ["mediapipe.solutions", "mediapipe.python.solutions"]
    for prefix in prefixes:
        try:
            pose  = importlib.import_module(f"{prefix}.pose")
            hands = importlib.import_module(f"{prefix}.hands")
            draw  = importlib.import_module(f"{prefix}.drawing_utils")
            logging.info(f"[mediapipe] loaded via '{prefix}'")
            return pose, hands, draw
        except (ImportError, ModuleNotFoundError):
            continue
    # Legacy attribute access (mediapipe <= 0.9)
    try:
        import mediapipe as mp
        logging.info("[mediapipe] loaded via legacy mp.solutions")
        return mp.solutions.pose, mp.solutions.hands, mp.solutions.drawing_utils
    except AttributeError:
        pass
    raise ImportError(
        "\nCannot import mediapipe Pose/Hands.\n"
        "Try:  pip uninstall mediapipe -y && pip install mediapipe==0.10.18\n"
        "Then: python tools/probe_mediapipe.py\n"
    )


mp_pose, mp_hands, mp_drawing = _import_mp_modules()

# Expose connection sets for infer.py webcam overlay
POSE_CONNECTIONS = mp_pose.POSE_CONNECTIONS
HAND_CONNECTIONS = mp_hands.HAND_CONNECTIONS
mp_holistic      = None   # Holistic removed in 0.10.13+ - kept as None for compat

# === Constants ================================================================

POSE_N   = 33
HAND_N   = 21
TOTAL_N  = POSE_N + HAND_N * 2   # 75
FEAT_DIM = TOTAL_N * 3           # 225

POSE_LEFT_SHOULDER  = 11
POSE_RIGHT_SHOULDER = 12
LEFT_HAND_START     = POSE_N           # 33
RIGHT_HAND_START    = POSE_N + HAND_N  # 54


# === Frame-level extraction ===================================================

def _lm_to_array(lm_obj, n: int) -> np.ndarray:
```

```
    if lm_obj is None:
        return np.zeros((n, 3), dtype=np.float32)
    return np.array([[lm.x, lm.y, lm.z] for lm in lm_obj.landmark], dtype=np.float32)


def extract_frame_keypoints(pose_results, hands_results) -> np.ndarray:
    """
    Build (75, 3) array for one frame from separate pose + hands results.

    Hand label convention:
      MediaPipe reports handedness from the *camera* perspective (mirrored).
      'Right' in the result = signer's LEFT hand, and vice-versa.
      We swap so our indices match the signer's anatomy.
    """
    pose = _lm_to_array(getattr(pose_results, 'pose_landmarks', None), POSE_N)
    lh   = np.zeros((HAND_N, 3), dtype=np.float32)
    rh   = np.zeros((HAND_N, 3), dtype=np.float32)

    if hands_results.multi_hand_landmarks:
        for lm_list, handedness in zip(
            hands_results.multi_hand_landmarks,
            hands_results.multi_handedness,
        ):
            label = handedness.classification[0].label   # 'Left' or 'Right'
            arr   = _lm_to_array(lm_list, HAND_N)
            if label == 'Right':
                lh = arr   # camera-right = signer's left
            else:
                rh = arr   # camera-left  = signer's right

    return np.concatenate([pose, lh, rh], axis=0)   # (75, 3)


def extract_frame_visibility(pose_results) -> np.ndarray:
    """Pose visibility scores. Shape: (33,)."""
    pl = getattr(pose_results, 'pose_landmarks', None)
    if pl is None:
        return np.zeros(POSE_N, dtype=np.float32)
    return np.array([lm.visibility for lm in pl.landmark], dtype=np.float32)


# === Normalization ============================================================

def normalize_sequence(seq: np.ndarray) -> np.ndarray:
    """
    Translate to shoulder-midpoint origin; scale by shoulder width.
    Input/output shape: (T, 75, 3).
    """
```

```
    l_sh = seq[:, POSE_LEFT_SHOULDER,  :]
    r_sh = seq[:, POSE_RIGHT_SHOULDER, :]
    mid  = (l_sh + r_sh) / 2.0
    dist = np.linalg.norm(r_sh - l_sh, axis=1, keepdims=True)
    dist = np.where(dist < 1e-6, 1.0, dist)
    return ((seq - mid[:, None, :]) / dist[:, None, :]).astype(np.float32)


# === Per-video processing ============================================

def process_video(video_path: str, detectors: dict, normalize: bool = True) -> dict:
    """
    Run Pose + Hands on every frame.

    Args:
        video_path: path to video file
        detectors:  {'pose': <Pose context>, 'hands': <Hands context>}
        normalize:  apply shoulder-based normalization

    Returns:
        dict with 'keypoints' (T,75,3), 'visibility' (T,33),
        'num_frames', 'fps', 'hands_present' - or {'keypoints': None} on error.
    """
    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        logging.warning(f"Cannot open: {video_path}")
        return {'keypoints': None}

    fps       = cap.get(cv2.CAP_PROP_FPS) or 30.0
    pose_det  = detectors['pose']
    hands_det = detectors['hands']
    kp_buf    = []
    vis_buf   = []
    lh_cnt    = rh_cnt = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        rgb.flags.writeable = False

        p_res = pose_det.process(rgb)
        h_res = hands_det.process(rgb)

        kp_buf.append(extract_frame_keypoints(p_res, h_res))
        vis_buf.append(extract_frame_visibility(p_res))
```

```
        if h_res.multi_hand_landmarks and h_res.multi_handedness:
            for hd in h_res.multi_handedness:
                if hd.classification[0].label == 'Right':
                    lh_cnt += 1
                else:
                    rh_cnt += 1

    cap.release()
    if not kp_buf:
        return {'keypoints': None}

    keypoints  = np.stack(kp_buf,  axis=0)   # (T, 75, 3)
    visibility = np.stack(vis_buf, axis=0)   # (T, 33)
    T          = keypoints.shape[0]

    if normalize:
        keypoints = normalize_sequence(keypoints)

    return {
        'keypoints':     keypoints,
        'visibility':    visibility,
        'num_frames':    T,
        'fps':           fps,
        'hands_present': (lh_cnt / T, rh_cnt / T),
    }


# === Batch extraction ============================================================

def extract_all_videos(
    video_dir:        str,
    output_dir:       str,
    normalize:        bool = True,
    model_complexity: int  = 2,
    overwrite:        bool = False,
) -> dict:
    """Process all videos; save .npy keypoints mirroring the folder structure."""
    video_dir  = Path(video_dir)
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    video_exts = {'.mp4', '.avi', '.mov', '.mkv', '.webm',
                  '.MP4', '.AVI', '.MOV', '.MKV'}

    import re as _re
    def _num_key(p):
        parts = _re.split(r'(\d+)', p.stem)
        return [int(x) if x.isdigit() else x.lower() for x in parts]
```

```
videos = sorted(
    [p for p in video_dir.rglob('*') if p.suffix in video_exts],
    key=_num_key,
)

logging.info(f"Found {len(videos)} videos in '{video_dir}'")
if not videos:
    raise RuntimeError(f"No videos found in {video_dir}")

stats = {
    'total': len(videos), 'processed': 0, 'failed': 0, 'skipped': 0,
    'seq_lengths': [], 'fps_values': [], 'lh_presence': [], 'rh_presence': [],
}

# Hands model_complexity only accepts 0 or 1
hands_complexity = min(model_complexity, 1)

with mp_pose.Pose(
    static_image_mode=False, model_complexity=model_complexity,
    smooth_landmarks=True, enable_segmentation=False,
    min_detection_confidence=0.5, min_tracking_confidence=0.5,
) as pose_det, mp_hands.Hands(
    static_image_mode=False, max_num_hands=2,
    model_complexity=hands_complexity,
    min_detection_confidence=0.5, min_tracking_confidence=0.5,
) as hands_det:

    detectors = {'pose': pose_det, 'hands': hands_det}

    for vp in tqdm(videos, desc="Extracting keypoints", unit="video"):
        rel   = vp.relative_to(video_dir)
        out_p = output_dir / rel.with_suffix('.npy')
        out_v = output_dir / rel.parent / (rel.stem + '_vis.npy')
        out_p.parent.mkdir(parents=True, exist_ok=True)

        if out_p.exists() and not overwrite:
            stats['seq_lengths'].append(np.load(str(out_p), mmap_mode='r').shape[0])
            stats['skipped'] += 1
            continue

        result = process_video(str(vp), detectors, normalize=normalize)

        if result['keypoints'] is None:
            logging.error(f"FAILED: {vp}")
            stats['failed'] += 1
            continue
```

```
            np.save(str(out_p), result['keypoints'])
            np.save(str(out_v), result['visibility'])
            stats['processed']    += 1
            stats['seq_lengths'].append(result['num_frames'])
            stats['fps_values'].append(result['fps'])
            stats['lh_presence'].append(result['hands_present'][0])
            stats['rh_presence'].append(result['hands_present'][1])

    sl = stats['seq_lengths']
    summary = {'total': stats['total'], 'processed': stats['processed'],
               'failed': stats['failed'], 'skipped': stats['skipped']}
    if sl:
        summary.update({
            'seq_len_min':      int(np.min(sl)),
            'seq_len_max':      int(np.max(sl)),
            'seq_len_mean':     float(np.mean(sl)),
            'seq_len_std':      float(np.std(sl)),
            'seq_len_p50':      float(np.percentile(sl, 50)),
            'seq_len_p95':      float(np.percentile(sl, 95)),
            'lh_mean_presence': float(np.mean(stats['lh_presence'])) if stats['lh_presence'] el
            'rh_mean_presence': float(np.mean(stats['rh_presence'])) if stats['rh_presence'] el
        })

    with open(output_dir / 'extraction_stats.json', 'w') as f:
        json.dump(summary, f, indent=2)

    print("\n" + "="*55)
    print("  Keypoint Extraction Summary")
    print("="*55)
    for k, v in summary.items():
        print(f"  {k:<28}: {v}")
    print("="*55 + "\n")
    return summary


# === CLI ======================================================================

def main():
    parser = argparse.ArgumentParser(
        description="Extract MediaPipe Pose+Hands keypoints from MSL videos"
    )
    parser.add_argument('--video_dir',    required=True)
    parser.add_argument('--output_dir',   required=True)
    parser.add_argument('--no_normalize', action='store_true')
    parser.add_argument('--complexity',   type=int, default=2, choices=[0, 1, 2])
    parser.add_argument('--overwrite',    action='store_true')
    parser.add_argument('--log_file',     default=None)
    args = parser.parse_args()
```

```
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)-8s | %(message)s",
        handlers=[
            logging.StreamHandler(),
            *([] if not args.log_file else [logging.FileHandler(args.log_file)]),
        ],
    )
    extract_all_videos(
        video_dir        = args.video_dir,
        output_dir       = args.output_dir,
        normalize        = not args.no_normalize,
        model_complexity = args.complexity,
        overwrite        = args.overwrite,
    )

if __name__ == '__main__':
    main()
```

1.6 infer.py

[6]:
```
from IPython.display import Markdown

file_path = './src/infer.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
infer.py - Single-video / real-time inference for MSL Recognition

Modes

  --video   : run on one video file, print top-k predictions
  --webcam  : live webcam inference (press Q to quit, R to reset buffer)
  --batch   : run on a directory of videos, produce CSV

Usage

  python src/infer.py \
      --checkpoint results/exp_A_bilstm/checkpoints/best.pth \
```

```
      --config     config/config.yaml \
      --video      data/videos/sample.mp4 \
      --top_k      5

  python src/infer.py \
      --checkpoint results/exp_A_bilstm/checkpoints/best.pth \
      --config     config/config.yaml \
      --webcam 0

  python src/infer.py \
      --checkpoint results/exp_A_bilstm/checkpoints/best.pth \
      --config     config/config.yaml \
      --batch      data/videos/new_samples/ \
      --output     results/batch_predictions.csv
"""

import argparse
import csv
import sys
import time
from pathlib import Path
from typing import List, Optional, Tuple

import cv2
import numpy as np
import torch
import torch.nn.functional as F
import yaml

sys.path.insert(0, str(Path(__file__).parent))

# Import shared mediapipe objects and helpers from extract_keypoints.
# mp_pose, mp_hands, mp_drawing are the live module handles.
# mp_holistic is None (removed in mediapipe >= 0.10.13) - not used here.
from extract_keypoints import (
    mp_pose, mp_hands, mp_drawing,
    POSE_CONNECTIONS, HAND_CONNECTIONS,
    process_video,
    extract_frame_keypoints,
    normalize_sequence,
    FEAT_DIM,
)
from models import build_model
from utils  import get_device, load_label_map, set_seed, get_logger


#    Model loading
```

```python
def load_model(checkpoint_path: str, config_path: str, device: torch.device):
    with open(config_path) as f:
        cfg = yaml.safe_load(f)

    ckpt        = torch.load(checkpoint_path, map_location=device)
    model_type  = ckpt.get('model_type', 'bilstm')
    num_classes = ckpt.get('num_classes')

    if num_classes is None:
        raise ValueError("Checkpoint missing 'num_classes'. Re-train to regenerate.")

    model = build_model(model_type, cfg, num_classes).to(device)
    model.load_state_dict(ckpt['state_dict'])
    model.eval()
    return model, model_type, cfg


#   Sequence pre-processing

def preprocess_sequence(
    seq:         np.ndarray,   # (T, 75, 3)
    max_seq_len: int,
    model_type:  str,
) -> torch.Tensor:
    """Centre-crop or zero-pad to max_seq_len; reshape for model input."""
    T = seq.shape[0]
    if T > max_seq_len:
        start = (T - max_seq_len) // 2
        seq   = seq[start: start + max_seq_len]
        T     = max_seq_len
    if T < max_seq_len:
        pad = np.zeros((max_seq_len - T, 75, 3), dtype=np.float32)
        seq = np.concatenate([seq, pad], axis=0)

    if model_type == 'stgcn':
        # (T, 75, 3) → (1, 3, T, 75)
        return torch.from_numpy(seq).permute(2, 0, 1).unsqueeze(0)
    else:
        # (T, 75, 3) → (1, T, 225)
        return torch.from_numpy(seq.reshape(max_seq_len, -1)).unsqueeze(0)


def build_pad_mask(real_len: int, max_len: int, device: torch.device) -> torch.Tensor:
    mask = torch.zeros(1, max_len, dtype=torch.bool, device=device)
    if real_len < max_len:
        mask[0, real_len:] = True
    return mask
```

```python
def predict_tensor(
    model,
    tensor:      torch.Tensor,
    model_type:  str,
    real_len:    int,
    max_seq_len: int,
    device:      torch.device,
    top_k:       int = 5,
) -> List[Tuple[int, float]]:
    """One forward pass → list of (class_idx, probability) sorted by prob desc."""
    tensor = tensor.to(device)
    mask   = build_pad_mask(real_len, max_seq_len, device)
    length = torch.tensor([real_len], dtype=torch.long, device=device)

    with torch.no_grad():
        if model_type == 'bilstm':
            logits = model(tensor, lengths=length, mask=mask)
        elif model_type == 'transformer':
            logits = model(tensor, mask=mask)
        else:  # stgcn
            logits = model(tensor)

    probs              = F.softmax(logits, dim=1)[0]
    k                  = min(top_k, probs.numel())
    topk_probs, topk_idxs = probs.topk(k)
    return list(zip(topk_idxs.cpu().tolist(), topk_probs.cpu().tolist()))


#   Helper: open Pose + Hands detectors

def _open_detectors(model_complexity: int = 1):
    """
    Return a context-managed pair of (pose, hands) detectors.
    Hands model_complexity is capped at 1 (mediapipe constraint).
    """
    return (
        mp_pose.Pose(
            static_image_mode=False,
            model_complexity=model_complexity,
            smooth_landmarks=True,
            enable_segmentation=False,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        ),
        mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=2,
```

```
            model_complexity=min(model_complexity, 1),
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        ),
    )


#   Single-video mode

def infer_video(
    video_path: str,
    model,
    model_type: str,
    cfg:        dict,
    idx2label:  dict,
    device:     torch.device,
    top_k:      int = 5,
) -> List[Tuple[str, float]]:
    max_seq_len = cfg['data']['max_seq_len']

    pose_det, hands_det = _open_detectors(model_complexity=2)
    try:
        result = process_video(
            video_path,
            detectors={'pose': pose_det, 'hands': hands_det},
            normalize=True,
        )
    finally:
        pose_det.close()
        hands_det.close()

    if result['keypoints'] is None:
        print(f"[ERROR] Could not extract keypoints from: {video_path}")
        return []

    seq      = result['keypoints']
    real_len = min(seq.shape[0], max_seq_len)
    tensor   = preprocess_sequence(seq, max_seq_len, model_type)
    preds    = predict_tensor(model, tensor, model_type, real_len, max_seq_len, device, top_k)

    print(f"\n{' '*55}")
    print(f"  Video : {Path(video_path).name}")
    print(f"  Frames: {result['num_frames']}  FPS: {result['fps']:.1f}")
    print(f"  Hands : L={result['hands_present'][0]*100:.0f}%  "
          f"R={result['hands_present'][1]*100:.0f}%")
    print(f"  Top-{top_k} predictions:")
    for rank, (idx, prob) in enumerate(preds, start=1):
        label = idx2label.get(idx, str(idx))
```

```
        bar   = ' ' * int(prob * 30)
        print(f"    {rank}. [{prob*100:5.1f}%] {bar:<30}  {label}")
    print(f"{' '*55}\n")

    return [(idx2label.get(idx, str(idx)), prob) for idx, prob in preds]


#   Batch mode

def infer_batch(
    video_dir:  str,
    model,
    model_type: str,
    cfg:        dict,
    idx2label:  dict,
    device:     torch.device,
    output_csv: str,
    top_k:      int = 1,
):
    video_dir  = Path(video_dir)
    video_exts = {'.mp4', '.avi', '.mov', '.mkv', '.webm',
                  '.MP4', '.AVI', '.MOV', '.MKV'}
    videos     = sorted([p for p in video_dir.rglob('*') if p.suffix in video_exts])

    if not videos:
        print(f"No videos found in {video_dir}")
        return

    rows        = []
    max_seq_len = cfg['data']['max_seq_len']

    pose_det, hands_det = _open_detectors(model_complexity=2)
    try:
        detectors = {'pose': pose_det, 'hands': hands_det}
        for vp in videos:
            result = process_video(str(vp), detectors, normalize=True)
            if result['keypoints'] is None:
                rows.append({'video': str(vp), 'pred_1': 'FAILED', 'conf_1': 0.0})
                print(f"  {vp.name:<40} → FAILED")
                continue

            seq      = result['keypoints']
            real_len = min(seq.shape[0], max_seq_len)
            tensor   = preprocess_sequence(seq, max_seq_len, model_type)
            preds    = predict_tensor(model, tensor, model_type, real_len,
                                      max_seq_len, device, top_k=top_k)
            row = {'video': str(vp)}
            for i, (idx, prob) in enumerate(preds, start=1):
```

```
                row[f'pred_{i}'] = idx2label.get(idx, str(idx))
                row[f'conf_{i}'] = round(prob, 4)
            rows.append(row)
            print(f"  {vp.name:<40} → {row.get('pred_1', '?')}")
    finally:
        pose_det.close()
        hands_det.close()

    Path(output_csv).parent.mkdir(parents=True, exist_ok=True)
    fieldnames = ['video'] + [
        f'{t}_{i}' for i in range(1, top_k + 1) for t in ('pred', 'conf')
    ]
    with open(output_csv, 'w', newline='', encoding='utf-8-sig') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames, extrasaction='ignore')
        writer.writeheader()
        writer.writerows(rows)
    print(f"\nBatch predictions saved → {output_csv}")


#   Webcam (live) mode

def infer_webcam(
    cam_idx:    int,
    model,
    model_type: str,
    cfg:        dict,
    idx2label:  dict,
    device:     torch.device,
    buffer_sec: float = 2.5,
):
    """
    Accumulate `buffer_sec` seconds of frames → classify → overlay prediction.
    Press Q to quit, R to reset the keypoint buffer.
    """
    max_seq_len = cfg['data']['max_seq_len']
    cap         = cv2.VideoCapture(cam_idx)
    fps         = cap.get(cv2.CAP_PROP_FPS) or 30.0
    buf_frames  = int(fps * buffer_sec)

    kp_buffer = []
    last_pred = "Collecting frames..."
    last_conf = 0.0

    pose_det, hands_det = _open_detectors(model_complexity=1)
    try:
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
```

```
        break

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    rgb.flags.writeable = False

    p_res = pose_det.process(rgb)
    h_res = hands_det.process(rgb)

    # Accumulate keypoints
    kp = extract_frame_keypoints(p_res, h_res)
    kp_buffer.append(kp)

    # Draw landmarks
    if p_res.pose_landmarks:
        mp_drawing.draw_landmarks(frame, p_res.pose_landmarks, POSE_CONNECTIONS)
    if h_res.multi_hand_landmarks:
        for lm in h_res.multi_hand_landmarks:
            mp_drawing.draw_landmarks(frame, lm, HAND_CONNECTIONS)

    # Classify when buffer is full
    if len(kp_buffer) >= buf_frames:
        seq      = normalize_sequence(np.stack(kp_buffer, axis=0))
        kp_buffer.clear()
        real_len = min(seq.shape[0], max_seq_len)
        tensor   = preprocess_sequence(seq, max_seq_len, model_type)
        preds    = predict_tensor(model, tensor, model_type, real_len,
                                  max_seq_len, device, top_k=1)
        if preds:
            last_pred = idx2label.get(preds[0][0], '?')
            last_conf = preds[0][1]

    # Overlay: black bar at bottom with prediction text
    h, w = frame.shape[:2]
    cv2.rectangle(frame, (0, h - 60), (w, h), (0, 0, 0), -1)
    cv2.putText(frame, f"{last_pred}  ({last_conf*100:.1f}%)",
                (10, h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 80), 2)
    # Buffer fill progress bar
    fill = int(len(kp_buffer) / max(buf_frames, 1) * w)
    cv2.rectangle(frame, (0, h - 62), (fill, h - 58), (0, 200, 255), -1)

    cv2.imshow('MSL Recognition  [Q=quit  R=reset]', frame)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('r'):
        kp_buffer.clear()
        last_pred = "Buffer reset"
        last_conf = 0.0
```

```
    finally:
        pose_det.close()
        hands_det.close()
        cap.release()
        cv2.destroyAllWindows()


#   Entry point

def main():
    parser = argparse.ArgumentParser(description='MSL Sign Language Inference')
    parser.add_argument('--checkpoint', required=True, help='Path to .pth checkpoint')
    parser.add_argument('--config',     default='config/config.yaml')

    group = parser.add_mutually_exclusive_group(required=True)
    group.add_argument('--video',  help='Path to a single video file')
    group.add_argument('--webcam', type=int, metavar='CAM_IDX',
                       help='Webcam device index for live inference')
    group.add_argument('--batch',  help='Directory of videos for batch inference')

    parser.add_argument('--top_k',     type=int, default=5)
    parser.add_argument('--output',    default='results/batch_predictions.csv')
    parser.add_argument('--label_map', default=None)
    args = parser.parse_args()

    logger = get_logger('infer')
    device = get_device()

    model, model_type, cfg = load_model(args.checkpoint, args.config, device)
    logger.info(f"Loaded {model_type} from {args.checkpoint}")

    lm_path          = args.label_map or cfg['data']['label_map_file']
    label2idx, idx2label = load_label_map(lm_path)

    if args.video:
        infer_video(args.video, model, model_type, cfg, idx2label, device,
                    top_k=args.top_k)
    elif args.webcam is not None:
        infer_webcam(args.webcam, model, model_type, cfg, idx2label, device)
    elif args.batch:
        infer_batch(args.batch, model, model_type, cfg, idx2label,
                    device, output_csv=args.output, top_k=args.top_k)


if __name__ == '__main__':
    main()
```

1.7 plot_results_v2.py

[7]:
```python
from IPython.display import Markdown

file_path = './src/plot_results_v2.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
plot_results.py - Visualise training curves and compare experiments

Generates

  1. Training curves (loss + accuracy) per experiment
  2. Multi-experiment accuracy comparison bar chart (Validation & Test)
  3. Learning rate schedule overlay
  4. Cross-validation score distribution (if cv_summary.json present)

Usage

  # Plot one experiment
  python src/plot_results.py --exp results/exp01_bilstm

  # Compare multiple experiments (Generates comparison_val.png AND comparison_test.png)
  python src/plot_results.py \
      --exp results/exp_bilstm results/exp_transformer results/exp_stgcn \
      --output results/comparison_val.png \
      --output_test results/comparison_test.png
"""

# === FIX: Handle Jupyter's matplotlib backend conflict ===
import os
_mpl_backend = os.environ.get('MPLBACKEND')
if _mpl_backend and _mpl_backend not in [
    'gtk3agg', 'gtk3cairo', 'gtk4agg', 'gtk4cairo', 'macosx',
    'nbagg', 'notebook', 'qtagg', 'qtcairo', 'qt5agg', 'qt5cairo',
    'tkagg', 'tkcairo', 'webagg', 'wx', 'wxagg', 'wxcairo',
    'agg', 'cairo', 'pdf', 'pgf', 'ps', 'svg', 'template'
]:
    os.environ['MPLBACKEND'] = 'agg'
# === END FIX ===
```

```
import argparse
import json
from pathlib import Path
from typing import List, Optional

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np


PALETTE = ['#2196F3', '#F44336', '#4CAF50', '#FF9800', '#9C27B0',
           '#00BCD4', '#795548', '#607D8B']


def format_model_name(name: str) -> str:
    """Convert folder name to a readable model name."""
    for prefix in ['exp_', 'cv_', 'run_', 'test_']:
        if name.startswith(prefix):
            name = name[len(prefix):]

    acronyms = {
        'bilstm': 'BiLSTM', 'stgcn': 'ST-GCN', 'transformer': 'Transformer',
        'lstm': 'LSTM', 'gru': 'GRU', 'cnn': 'CNN', 'rnn': 'RNN',
        'gcn': 'GCN', 'tcn': 'TCN'
    }

    lower_name = name.lower()
    if lower_name in acronyms:
        return acronyms[lower_name]

    for key, val in acronyms.items():
        if lower_name.startswith(key):
            suffix = name[len(key):]
            if suffix.startswith('_'):
                suffix = suffix[1:]
            return f"{val} {suffix}" if suffix else val

    return name.replace('_', ' ').title()


def get_value(item: dict, key_options: list, default=None):
    """Try multiple key names and return the first one found."""
    for key in key_options:
        if key in item and item[key] is not None:
            return item[key]
    return default
```

```
def normalize_accuracy(value, default=0):
    """Convert accuracy to percentage (0-100) range."""
    if value is None:
        return default
    if value > 1.0:
        return value
    return value * 100


def load_history(exp_dir: Path) -> Optional[list]:
    h_path = exp_dir / 'history.json'
    if not h_path.exists():
        return None
    with open(h_path) as f:
        return json.load(f)


def load_cv_summary(exp_dir: Path) -> Optional[dict]:
    cv_path = exp_dir / 'cv_summary.json'
    if not cv_path.exists():
        return None
    with open(cv_path) as f:
        return json.load(f)


def plot_training_curves(exp_dir: Path, save_path: Optional[str] = None):
    history = load_history(exp_dir)
    if history is None:
        print(f"No history.json found in {exp_dir}")
        return

    epochs     = [get_value(h, ['epoch', 'Epoch', 'ep', 'step']) for h in history]
    train_loss = [get_value(h, ['loss', 'train_loss', 'trainLoss', 'Loss', 'train/loss']) for h
    val_loss   = [get_value(h, ['val_loss', 'valid_loss', 'valLoss', 'validation_loss', 'val/lo

    train_top1_raw = [get_value(h, ['top1', 'train_top1', 'train_acc', 'train_accuracy',
                                    'trainAcc', 'accuracy', 'train/acc', 'train/top1']) for h i
    train_top1 = [normalize_accuracy(v) for v in train_top1_raw]

    val_top1_raw = [get_value(h, ['val_top1', 'val_acc', 'val_accuracy', 'valAcc',
                                  'valid_acc', 'validation_accuracy', 'val/acc', 'val/top1'])
    val_top1 = [normalize_accuracy(v) for v in val_top1_raw]

    val_top5_raw = [get_value(h, ['val_top5', 'val_top_5', 'val_top5_acc', 'val/top5']) for h i
    val_top5 = [normalize_accuracy(v) for v in val_top5_raw]
```

```
lrs        = [get_value(h, ['lr', 'learning_rate', 'LR', 'train/lr']) for h in history]

valid_indices = [i for i, e in enumerate(epochs) if e is not None]
if not valid_indices:
    print(f"No valid epoch data in {exp_dir}")
    return

epochs     = [epochs[i] for i in valid_indices]
train_loss = [train_loss[i] if train_loss[i] is not None else float('nan') for i in valid_
val_loss   = [val_loss[i] if val_loss[i] is not None else float('nan') for i in valid_indi
train_top1 = [train_top1[i] for i in valid_indices]
val_top1   = [val_top1[i] for i in valid_indices]
val_top5   = [val_top5[i] for i in valid_indices]
lrs        = [lrs[i] for i in valid_indices]

has_train_loss = any(not (isinstance(v, float) and np.isnan(v)) for v in train_loss)
has_val_loss = any(not (isinstance(v, float) and np.isnan(v)) for v in val_loss)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle(f"Training Curves: {format_model_name(exp_dir.name)}", fontsize=14, fontweight

ax = axes[0]
if has_train_loss:
    ax.plot(epochs, train_loss, label='Train Loss', color=PALETTE[0], linewidth=2)
else:
    ax.text(0.5, 0.5, 'No train loss data', ha='center', va='center', transform=ax.transAx
if has_val_loss:
    ax.plot(epochs, val_loss, label='Val Loss', color=PALETTE[1], linewidth=2)
ax.set_xlabel('Epoch'); ax.set_ylabel('Loss')
ax.set_title('Loss'); ax.legend(); ax.grid(alpha=0.3)

ax = axes[1]
has_train_acc = any(v > 0 for v in train_top1)
has_val_acc = any(v > 0 for v in val_top1)

if has_train_acc:
    ax.plot(epochs, train_top1, label='Train Top-1', color=PALETTE[0], linewidth=2)
if has_val_acc:
    ax.plot(epochs, val_top1,   label='Val Top-1',   color=PALETTE[1], linewidth=2)
    valid_val_top1 = [(i, v) for i, v in enumerate(val_top1) if v > 0]
    if valid_val_top1:
        best_idx = max(valid_val_top1, key=lambda x: x[1])
        best_epoch = epochs[best_idx[0]]
        best_acc = best_idx[1]
        ax.axvline(best_epoch, color='grey', linestyle=':', alpha=0.7)
        ax.annotate(f'Best {best_acc:.1f}%', xy=(best_epoch, best_acc),
                    xytext=(best_epoch + 2, best_acc - 5),
                    arrowprops=dict(arrowstyle='->', color='black'), fontsize=9)
```

```
    has_val_top5 = any(v > 0 for v in val_top5)
    if has_val_top5:
        ax.plot(epochs, val_top5,   label='Val Top-5',   color=PALETTE[2], linewidth=2, linesty

    if not has_train_acc and not has_val_acc:
        ax.text(0.5, 0.5, 'No accuracy data', ha='center', va='center', transform=ax.transAxes)

    ax.set_xlabel('Epoch'); ax.set_ylabel('Accuracy (%)')
    ax.set_title('Accuracy'); ax.legend(); ax.grid(alpha=0.3)

    ax = axes[2]
    valid_lrs = [v for v in lrs if v is not None and v > 0]
    if valid_lrs:
        ax.semilogy(epochs, lrs, color=PALETTE[3], linewidth=2)
        ax.set_xlabel('Epoch'); ax.set_ylabel('Learning Rate (log)')
        ax.set_title('LR Schedule'); ax.grid(alpha=0.3)
    else:
        ax.text(0.5, 0.5, 'No LR data', ha='center', va='center', transform=ax.transAxes)

    plt.tight_layout()
    out = save_path or str(exp_dir / 'training_curves.png')
    plt.savefig(out, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"Training curves saved → {out}")


def plot_comparison(exp_dirs: List[Path], output: str, custom_names: Optional[List[str]] = None
    """Bar chart comparing best top1 across experiments for a specific split (val/test)."""
    names       = []
    best_top1s  = []
    cv_means    = []
    cv_stds     = []

    for i, exp_dir in enumerate(exp_dirs):
        # Read the precise evaluation metrics for the requested split
        metrics_path = exp_dir / 'evaluation' / f'metrics_{split}.json'

        # CV data is usually only relevant for validation
        cv = load_cv_summary(exp_dir) if split == 'val' else None

        if custom_names and i < len(custom_names):
            name = custom_names[i]
        else:
            name = format_model_name(exp_dir.name)

        found_data = False  # <--- ADD THIS FLAG
```

```
        # Priority 1: Use the exact metrics JSON
        if metrics_path.exists():
            with open(metrics_path) as f:
                metrics = json.load(f)
            names.append(name)
            best_top1s.append(metrics.get('top1_accuracy', 0.0) * 100)
            cv_means.append(None)
            cv_stds.append(None)
            found_data = True  # <--- MARK AS FOUND

        # Priority 2: Use CV summary (val only)
        elif cv:
            names.append(name)
            best_top1s.append(cv.get('best_top1', cv.get('mean_top1', 0)))
            cv_means.append(cv.get('mean_top1', 0))
            cv_stds.append(cv.get('std_top1', 0))
            found_data = True  # <--- MARK AS FOUND

        # Priority 3: Fallback to history.json (val only, less accurate)
        elif split == 'val':
            history = load_history(exp_dir)
            if history:
                val_top1s = []
                for h in history:
                    v = get_value(h, ['val_top1', 'val_acc', 'val_accuracy', 'valAcc',
                                      'valid_acc', 'validation_accuracy', 'val/acc', 'val/top1
                    val_top1s.append(normalize_accuracy(v))

                if any(v > 0 for v in val_top1s):
                    names.append(name)
                    best_top1s.append(max(val_top1s))
                    cv_means.append(None)
                    cv_stds.append(None)
                    found_data = True  # <--- MARK AS FOUND

        # REPLACE THE OLD BROKEN CHECK WITH THIS:
        if not found_data:
            print(f"[WARNING] No valid {split} data in {exp_dir}, skipping")

    if not names:
        print(f"No experiment data found for split '{split}'.")
        return

    x   = np.arange(len(names))
    fig, ax = plt.subplots(figsize=(max(8, len(names) * 2), 6))

    colors = [PALETTE[i % len(PALETTE)] for i in range(len(names))]
    bars   = ax.bar(x, best_top1s, color=colors, alpha=0.85, width=0.5)
```

```
    # CV error bars
    cv_label_added = False
    for i, (mean, std) in enumerate(zip(cv_means, cv_stds)):
        if mean is not None:
            label = 'CV Mean±Std' if not cv_label_added else None
            ax.errorbar(x[i], mean, yerr=std, fmt='D', color='black',
                        markersize=6, capsize=5, label=label)
            cv_label_added = True

    # Value labels on top of bars
    for bar, val in zip(bars, best_top1s):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.2,
                f'{val:.2f}%', ha='center', va='bottom', fontsize=10, fontweight='bold')

    ax.set_xticks(x)
    ax.set_xticklabels(names, rotation=0, ha='center', fontsize=12)
    ax.set_ylabel('Accuracy (%)', fontsize=12)
    ax.set_title(f'Model Comparison - {split.capitalize()} Top-1 Accuracy', fontsize=14, fontwe

    # Smart Y-axis limits: Zoom in if accuracy is very high (e.g., 99-100%)
    y_max = max(best_top1s)
    if y_max > 95.0:
        # Start Y-axis closer to the lowest score to make differences visible
        y_min = min(best_top1s) - 2.0
        ax.set_ylim(max(0, y_min), min(101, y_max + 1.5))
    else:
        ax.set_ylim(0, min(105, y_max + 10))

    ax.grid(axis='y', alpha=0.3)

    if cv_label_added:
        ax.legend(fontsize=10)

    plt.tight_layout()
    plt.savefig(output, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"{split.capitalize()} comparison chart saved → {output}")


def plot_cv_distribution(exp_dir: Path):
    cv = load_cv_summary(exp_dir)
    if cv is None:
        return

    scores = cv.get('fold_scores', cv.get('scores', cv.get('fold_accuracies', None)))
    if scores is None:
        return
```

```
    fig, ax = plt.subplots(figsize=(8, 4))

    folds = [f'Fold {i}' for i in range(len(scores))]
    bars  = ax.bar(folds, scores, color=PALETTE[:len(scores)], alpha=0.85)

    mean_top1 = cv.get('mean_top1', cv.get('mean_accuracy', np.mean(scores)))
    std_top1 = cv.get('std_top1', cv.get('std_accuracy', np.std(scores)))

    ax.axhline(mean_top1, color='red', linestyle='--', linewidth=2,
               label=f"Mean: {mean_top1:.1f}%")
    ax.fill_between(range(-1, len(scores)+1),
                    mean_top1 - std_top1,
                    mean_top1 + std_top1,
                    alpha=0.15, color='red', label=f"±Std: {std_top1:.1f}%")

    for bar, v in zip(bars, scores):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.3,
                f'{v:.1f}%', ha='center', fontsize=9, fontweight='bold')

    ax.set_ylim(0, min(105, max(scores) + 8))
    ax.set_ylabel('Top-1 Accuracy (%)')
    ax.set_title(f"Cross-Validation Distribution - {format_model_name(exp_dir.name)}", fontweig
    ax.legend(); ax.grid(axis='y', alpha=0.3)

    out = str(exp_dir / 'cv_distribution.png')
    plt.tight_layout()
    plt.savefig(out, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"CV distribution saved → {out}")


def main():
    parser = argparse.ArgumentParser(description='Plot MSL training results')
    parser.add_argument('--exp',         nargs='+', required=True,
                        help='One or more experiment directories')
    parser.add_argument('--names',       nargs='+', default=None,
                        help='Custom display names for experiments (optional)')
    parser.add_argument('--output',      default='results/comparison_val.png',
                        help='Output path for validation comparison chart')
    parser.add_argument('--output_test', default='results/comparison_test.png',
                        help='Output path for test comparison chart')
    args = parser.parse_args()

    exp_dirs = [Path(e) for e in args.exp]

    # Per-experiment curves
    for exp_dir in exp_dirs:
```

```
        if not exp_dir.exists():
            print(f"Warning: {exp_dir} not found, skipping")
            continue
        plot_training_curves(exp_dir)
        plot_cv_distribution(exp_dir)

    # Multi-experiment comparison (Now generates BOTH Val and Test!)
    if len(exp_dirs) > 1:
        valid = [e for e in exp_dirs if e.exists()]
        if valid:
            plot_comparison(valid, args.output,      args.names, split='val')
            plot_comparison(valid, args.output_test,  args.names, split='test')


if __name__ == '__main__':
    main()
```

1.8 prepare_data.py

[8]:
```
from IPython.display import Markdown

file_path = './src/prepare_data.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
prepare_data.py - One-shot data preparation for MSL Recognition

Steps performed

  1. Parse annotation file (TSV: normal_text <TAB> msl_gloss)
  2. Build label vocabulary and save label_map.json
  3. Match video files to annotation records (positional matching)
  4. Print dataset statistics
  5. Create stratified train / val / test splits  → splits.json
  6. Create stratified K-fold splits              → kfold_splits.json
  7. Verify keypoint files exist (if already extracted)

Usage

  python src/prepare_data.py --config config/config.yaml
```

```
  python src/prepare_data.py --config config/config.yaml --verify_keypoints
"""

import argparse
import json
import sys
from pathlib import Path

import yaml

sys.path.insert(0, str(Path(__file__).parent))
from utils import (
    get_logger,
    parse_annotation_file,
    build_label_vocabulary,
    save_label_map,
    match_videos_to_annotations,
    create_splits,
    create_kfold_splits,
    print_dataset_stats,
    set_seed,
)


def verify_keypoints(records: list, logger) -> dict:
    """Check which keypoint .npy files exist."""
    total   = len(records)
    found   = sum(1 for r in records if r.get('keypoint_path') and Path(r['keypoint_path']).ex
    missing = total - found
    logger.info(f"Keypoint verification: {found}/{total} found, {missing} missing")
    if missing > 0:
        logger.warning("Run extract_keypoints.py before training!")
        missing_list = [r.get('video_path', '?') for r in records
                        if not (r.get('keypoint_path') and Path(r['keypoint_path']).exists())]
        for p in missing_list[:10]:
            logger.warning(f"  Missing keypoints for: {p}")
        if len(missing_list) > 10:
            logger.warning(f"  ... and {len(missing_list)-10} more")
    return {'total': total, 'found': found, 'missing': missing}


def main():
    parser = argparse.ArgumentParser(description='Prepare MSL dataset: splits, label map, stat
    parser.add_argument('--config',           default='config/config.yaml')
    parser.add_argument('--verify_keypoints', action='store_true',
                        help='Check whether keypoint .npy files already exist')
    args = parser.parse_args()
```

```
with open(args.config) as f:
    cfg = yaml.safe_load(f)

dcfg = cfg['data']
set_seed(dcfg.get('seed', 42))

log_dir = Path(cfg['logging']['log_dir'])
log_dir.mkdir(parents=True, exist_ok=True)
logger = get_logger('prepare_data', log_file=str(log_dir / 'prepare_data.log'))

logger.info("=" * 60)
logger.info("MSL Data Preparation")
logger.info("=" * 60)

#   1. Parse annotations
logger.info(f"Parsing annotation file: {dcfg['annotation_file']}")
records = parse_annotation_file(dcfg['annotation_file'])
logger.info(f"Total annotation records: {len(records)}")

#   2. Label vocabulary
label2idx, idx2label = build_label_vocabulary(records)
num_classes = len(label2idx)
logger.info(f"Vocabulary built: {num_classes} unique classes")

Path(dcfg['label_map_file']).parent.mkdir(parents=True, exist_ok=True)
save_label_map(label2idx, idx2label, dcfg['label_map_file'])
logger.info(f"Label map saved → {dcfg['label_map_file']}")

#   3. Match videos
logger.info(f"Matching videos from: {dcfg['video_dir']}")
records = match_videos_to_annotations(
    video_dir    = dcfg['video_dir'],
    records      = records,
    keypoint_dir = dcfg.get('keypoint_dir'),
)

#   4. Stats
print_dataset_stats(records, label2idx, split='full')

# Research note for paper
logger.info(
    "NOTE: MSL4Emergency has ~1 video per sign class. "
    "The pipeline handles this via: "
    "(1) random (non-stratified) train/val/test split on raw videos, "
    "(2) heavy keypoint-space augmentation on training split only (20×), "
    "(3) val/test evaluated on original un-augmented keypoints. "
    "This is the standard approach for low-resource sign language research."
)
```

```
#   5. Train/val/test splits
logger.info("Creating random train/val/test splits (1-sample-per-class safe) …")
splits = create_splits(
    records     = records,
    label2idx   = label2idx,
    test_ratio  = dcfg.get('test_ratio', 0.15),
    val_ratio   = dcfg.get('val_ratio',  0.15),
    seed        = dcfg.get('seed', 42),
    output_path = dcfg['split_file'],
)

#   6. K-Fold splits
n_folds     = dcfg.get('n_folds', 5)
kfold_path  = str(Path(dcfg['split_file']).parent / 'kfold_splits.json')
logger.info(f"Creating {n_folds}-fold CV splits (standard KFold) …")
create_kfold_splits(
    records     = records,
    label2idx   = label2idx,
    n_folds     = n_folds,
    seed        = dcfg.get('seed', 42),
    output_path = kfold_path,
)

#   7. Verify keypoints (optional)
if args.verify_keypoints:
    logger.info("Verifying keypoint files …")
    kp_stats = verify_keypoints(records, logger)
    with open(log_dir / 'keypoint_verify.json', 'w') as f:
        json.dump(kp_stats, f, indent=2)

#   Summary
print("\n" + "=" * 60)
print("  Data preparation complete!")
print("=" * 60)
print(f"  Annotation records : {len(records)}")
print(f"  Unique classes     : {num_classes}")
print(f"  Train samples      : {len(splits['train'])}  (→ augmented ×20 in next step)")
print(f"  Val   samples      : {len(splits['val'])}   (original keypoints only)")
print(f"  Test  samples      : {len(splits['test'])}   (original keypoints only)")
print(f"  K-Fold splits      : {n_folds} folds  →  {kfold_path}")
print(f"  Label map          : {dcfg['label_map_file']}")
print(f"  Splits file        : {dcfg['split_file']}")
print(f"")
print(f"     Note: ~1 video/class → random (non-stratified) split used.")
print(f"     Val/test may not cover all {num_classes} classes - this is expected.")
print(f"     Evaluation metrics will reflect classes seen in each split.")
print("=" * 60 + "\n")
```

```
if __name__ == '__main__':
    main()
```

1.9 train.py

[9]:
```
from IPython.display import Markdown

file_path = './src/train.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
train.py - Training script for MSL Sign Language Recognition

Features

  - Automatic Mixed Precision (AMP) for RTX 3090 Ti
  - Early stopping with patience
  - Cosine / Step / Plateau LR scheduler
  - TensorBoard logging
  - Label smoothing
  - Class-weighted loss for imbalanced data
  - Top-k checkpoint saving
  - Gradient clipping

Usage

  python src/train.py \
      --config config/config.yaml \
      --model  bilstm \
      --exp    exp01_bilstm
"""

import argparse
import heapq
import json
import os
import sys
import time
```

```
from pathlib import Path

import torch
import torch.nn as nn
import yaml
from torch.amp import GradScaler, autocast
from torch.utils.tensorboard import SummaryWriter

sys.path.insert(0, str(Path(__file__).parent))
from dataset import build_dataloaders
from models  import build_model
from utils   import (
    get_logger, set_seed, get_device, count_parameters,
    parse_annotation_file, build_label_vocabulary,
    match_videos_to_annotations, load_splits,
    save_label_map, save_checkpoint, compute_class_weights,
)
from augment import RandomAugmentor


#   Focal Loss (optional)

class FocalLoss(nn.Module):
    def __init__(self, gamma: float = 2.0, weight=None, label_smoothing: float = 0.0):
        super().__init__()
        self.gamma   = gamma
        self.weight  = weight
        self.ls      = label_smoothing

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        ce_loss = nn.functional.cross_entropy(
            logits, targets, weight=self.weight,
            label_smoothing=self.ls, reduction='none'
        )
        pt  = torch.exp(-ce_loss)
        return ((1 - pt) ** self.gamma * ce_loss).mean()


#   Metrics

def compute_accuracy(logits: torch.Tensor, labels: torch.Tensor, top_k=(1, 5)):
    """Compute top-k accuracy."""
    results = {}
    with torch.no_grad():
        max_k = max(top_k)
        batch_size = labels.size(0)
        _, pred = logits.topk(min(max_k, logits.size(1)), dim=1, largest=True, sorted=True)
        pred = pred.t()
```

```
        correct = pred.eq(labels.view(1, -1).expand_as(pred))
        for k in top_k:
            if k <= logits.size(1):
                correct_k = correct[:k].reshape(-1).float().sum()
                results[f'top{k}'] = correct_k.item() / batch_size
            else:
                results[f'top{k}'] = 0.0
    return results


#    Train / Validate one epoch

def train_one_epoch(model, loader, optimizer, criterion, scaler, device, cfg, logger):
    model.train()
    total_loss = 0.0
    total_top1 = 0.0
    total_top5 = 0.0
    n_batches  = 0
    log_interval = cfg['logging'].get('log_interval', 20)

    for batch_idx, batch in enumerate(loader):
        kp     = batch['keypoints'].to(device, non_blocking=True)
        labels = batch['label'].to(device, non_blocking=True)
        mask   = batch['mask'].to(device, non_blocking=True)
        lengths= batch['length'].to(device, non_blocking=True)

        with autocast('cuda', enabled=cfg['training']['use_amp']):
            if hasattr(model, 'lstm'):
                logits = model(kp, lengths=lengths, mask=mask)
            elif hasattr(model, 'encoder'):
                logits = model(kp, mask=mask)
            else:
                logits = model(kp)
            loss = criterion(logits, labels)

        optimizer.zero_grad()
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(
            model.parameters(), cfg['training'].get('grad_clip', 1.0)
        )
        scaler.step(optimizer)
        scaler.update()

        acc = compute_accuracy(logits.detach(), labels)
        total_loss += loss.item()
        total_top1 += acc['top1']
        total_top5 += acc.get('top5', 0.0)
```

```
        n_batches  += 1

        if (batch_idx + 1) % log_interval == 0:
            logger.debug(
                f"  batch {batch_idx+1}/{len(loader)}  "
                f"loss={loss.item():.4f}  top1={acc['top1']*100:.1f}%"
            )

    return {
        'loss': total_loss / n_batches,
        'top1': total_top1 / n_batches,
        'top5': total_top5 / n_batches,
    }


@torch.no_grad()
def validate(model, loader, criterion, device, cfg):
    model.eval()
    total_loss = 0.0
    total_top1 = 0.0
    total_top5 = 0.0
    n_batches  = 0

    for batch in loader:
        kp     = batch['keypoints'].to(device, non_blocking=True)
        labels = batch['label'].to(device, non_blocking=True)
        mask   = batch['mask'].to(device, non_blocking=True)
        lengths= batch['length'].to(device, non_blocking=True)

        with autocast('cuda', enabled=cfg['training']['use_amp']):
            if hasattr(model, 'lstm'):
                logits = model(kp, lengths=lengths, mask=mask)
            elif hasattr(model, 'encoder'):
                logits = model(kp, mask=mask)
            else:
                logits = model(kp)
            loss = criterion(logits, labels)

        acc = compute_accuracy(logits, labels)
        total_loss += loss.item()
        total_top1 += acc['top1']
        total_top5 += acc.get('top5', 0.0)
        n_batches  += 1

    return {
        'loss': total_loss / n_batches,
        'top1': total_top1 / n_batches,
        'top5': total_top5 / n_batches,
```

```
    }


#    Main training loop

def train(args):
    # Config
    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    tcfg = cfg['training']
    dcfg = cfg['data']
    set_seed(tcfg['seed'])

    # Experiment directories
    exp_dir  = Path('results') / args.exp
    ckpt_dir = exp_dir / 'checkpoints'
    log_dir  = exp_dir / 'logs'
    exp_dir.mkdir(parents=True, exist_ok=True)
    ckpt_dir.mkdir(parents=True, exist_ok=True)
    log_dir.mkdir(parents=True, exist_ok=True)

    # Save config snapshot
    with open(exp_dir / 'config_used.yaml', 'w') as f:
        yaml.dump(cfg, f)

    logger = get_logger('train', log_file=str(log_dir / 'train.log'))
    logger.info(f"Experiment: {args.exp}  Model: {args.model}")

    device = get_device()

    #   Data
    records              = parse_annotation_file(dcfg['annotation_file'])
    label2idx, idx2label = build_label_vocabulary(records)
    num_classes          = len(label2idx)
    logger.info(f"Classes: {num_classes}")

    save_label_map(label2idx, idx2label, dcfg['label_map_file'])

    records = match_videos_to_annotations(
        dcfg['video_dir'], records, dcfg['keypoint_dir']
    )

    #   Load splits from all-class augmented manifest
    # The new design: augmented_manifest.json has keys 'train', 'val', 'test'
    # where ALL 556 classes are present in every split.  This is the only
    # correct approach when there is 1 original video per class.
    aug_manifest_path = Path(dcfg['augmented_dir']) / 'augmented_manifest.json'
```

```
if not aug_manifest_path.exists():
    logger.error(
        f"Augmented manifest not found: {aug_manifest_path}\n"
        "Run:  bash scripts/03_augment_data.sh"
    )
    raise FileNotFoundError(str(aug_manifest_path))

with open(aug_manifest_path, encoding='utf-8') as f:
    manifest = json.load(f)

# Support both new dict format {'train':[], 'val':[], 'test':[]}
# and old flat-list format (legacy, just in case)
if isinstance(manifest, dict) and 'train' in manifest:
    train_records = manifest['train']
    val_records   = manifest['val']
    test_records  = manifest['test']
    logger.info(
        f"Using all-class manifest: "
        f"train={len(train_records)}, val={len(val_records)}, test={len(test_records)}"
    )
else:
    # Old flat-list format fallback (pre-fix manifest)
    logger.warning(
        "Old flat manifest format detected. Re-run 03_augment_data.sh "
        "to regenerate with the all-class design."
    )
    train_records = manifest
    splits        = load_splits(dcfg['split_file'])
    val_records   = [records[i] for i in splits['val']]
    test_records  = [records[i] for i in splits['test']]

# No on-the-fly augmentation needed - training data is already augmented
augmentor = None

train_loader, val_loader, test_loader = build_dataloaders(
    train_records, val_records, test_records,
    label2idx, cfg,
    augmentor  = augmentor,
    model_type = args.model,
)

#   Model
model = build_model(args.model, cfg, num_classes).to(device)
n_params = count_parameters(model)
logger.info(f"Model: {args.model}  Params: {n_params:,}")

#   Loss
```

```
# All classes have equal aug counts → uniform weights; still pass for API compat
class_weights = compute_class_weights(
    train_records, label2idx, device
) if args.weighted_loss else None

if tcfg.get('loss', 'cross_entropy') == 'focal':
    criterion = FocalLoss(
        gamma          = tcfg.get('focal_gamma', 2.0),
        weight         = class_weights,
        label_smoothing= tcfg.get('label_smoothing', 0.0),
    )
else:
    criterion = nn.CrossEntropyLoss(
        weight          = class_weights,
        label_smoothing = tcfg.get('label_smoothing', 0.0),
    )

#   Optimiser
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr           = tcfg['learning_rate'],
    weight_decay = tcfg['weight_decay'],
)

#   LR Scheduler
sched_name = tcfg.get('scheduler', 'cosine')
if sched_name == 'cosine':
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max   = tcfg.get('cosine_T_max', tcfg['num_epochs']),
        eta_min = tcfg.get('cosine_eta_min', 1e-6),
    )
elif sched_name == 'step':
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.5)
else:
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='max', patience=10, factor=0.5
    )

# Warmup wrapper - use ChainedScheduler pattern to avoid SequentialLR
# internal epoch-passing deprecation warning
warmup_epochs = tcfg.get('warmup_epochs', 0)
if warmup_epochs > 0:
    warmup = torch.optim.lr_scheduler.LinearLR(
        optimizer, start_factor=0.01, end_factor=1.0, total_iters=warmup_epochs
    )
    scheduler = torch.optim.lr_scheduler.SequentialLR(
        optimizer, schedulers=[warmup, scheduler], milestones=[warmup_epochs]
```

```
    )

#   AMP scaler
scaler = GradScaler('cuda', enabled=tcfg['use_amp'])

#   TensorBoard
writer = SummaryWriter(log_dir=str(log_dir)) if cfg['logging']['use_tensorboard'] else None

#   Training loop
# Start at -1 so the very first epoch always triggers a checkpoint save,
# even when val accuracy is 0%.  Without this, a model that never exceeds
# 0% accuracy (common early in training with 556 classes) never writes
# best.pth and the evaluation script fails with "checkpoint not found".
best_val_top1  = -1.0
patience_count = 0
patience       = tcfg['patience']
save_top_k     = tcfg.get('save_top_k', 3)
top_k_heap     = []  # min-heap of (val_top1, epoch, filepath)
history        = []

logger.info("Starting training…")
for epoch in range(1, tcfg['num_epochs'] + 1):
    t0 = time.time()

    train_metrics = train_one_epoch(
        model, train_loader, optimizer, criterion, scaler, device, cfg, logger
    )
    val_metrics = validate(model, val_loader, criterion, device, cfg)

    # Scheduler step
    if isinstance(scheduler, torch.optim.lr_scheduler.ReduceLROnPlateau):
        scheduler.step(val_metrics['top1'])
    else:
        scheduler.step()

    lr  = optimizer.param_groups[0]['lr']
    dur = time.time() - t0

    logger.info(
        f"Epoch {epoch:3d}/{tcfg['num_epochs']} | "
        f"train loss={train_metrics['loss']:.4f} top1={train_metrics['top1']*100:.2f}% | "
        f"val loss={val_metrics['loss']:.4f} top1={val_metrics['top1']*100:.2f}% "
        f"top5={val_metrics['top5']*100:.2f}% | "
        f"lr={lr:.2e} | {dur:.0f}s"
    )

    # TensorBoard
    if writer:
```

```
    writer.add_scalar('Loss/train', train_metrics['loss'], epoch)
    writer.add_scalar('Loss/val',   val_metrics['loss'],   epoch)
    writer.add_scalar('Acc/train_top1', train_metrics['top1'], epoch)
    writer.add_scalar('Acc/val_top1',   val_metrics['top1'],   epoch)
    writer.add_scalar('Acc/val_top5',   val_metrics['top5'],   epoch)
    writer.add_scalar('LR', lr, epoch)

history.append({
    'epoch': epoch,
    **{f'train_{k}': v for k, v in train_metrics.items()},
    **{f'val_{k}':   v for k, v in val_metrics.items()},
    'lr': lr,
})

# Checkpoint (save top-k)
ckpt_path = ckpt_dir / f"epoch{epoch:03d}_val{val_metrics['top1']:.4f}.pth"
ckpt_state = {
    'epoch':      epoch,
    'model_type': args.model,
    'state_dict': model.state_dict(),
    'optimizer':  optimizer.state_dict(),
    'val_top1':   val_metrics['top1'],
    'num_classes': num_classes,
    'config':     cfg,
}

is_best = val_metrics['top1'] > best_val_top1
if is_best:
    best_val_top1  = val_metrics['top1']
    patience_count = 0
    save_checkpoint(ckpt_state, str(ckpt_path),
                    is_best=True,
                    best_filepath=str(ckpt_dir / 'best.pth'))
    logger.info(f"    New best val top-1: {best_val_top1*100:.2f}%")
else:
    patience_count += 1

# Top-k heap management
heapq.heappush(top_k_heap, (val_metrics['top1'], epoch, str(ckpt_path)))
if len(top_k_heap) > save_top_k:
    worst_score, worst_epoch, worst_path = heapq.heappop(top_k_heap)
    if Path(worst_path).exists() and worst_path != str(ckpt_dir / 'best.pth'):
        Path(worst_path).unlink()

if not is_best:
    save_checkpoint(ckpt_state, str(ckpt_path))

# Early stopping
```

```
            if patience_count >= patience:
                logger.info(f"Early stopping at epoch {epoch} (patience={patience})")
                break

        # Save training history
        with open(exp_dir / 'history.json', 'w') as f:
            json.dump(history, f, indent=2)

        if writer:
            writer.close()

        logger.info(f"Training complete. Best val top-1: {best_val_top1*100:.2f}%")
        logger.info(f"Best checkpoint: {ckpt_dir / 'best.pth'}")
        return best_val_top1


    #   Entry point

    def main():
        parser = argparse.ArgumentParser(description='Train MSL Sign Language Recognition Model')
        parser.add_argument('--config',        default='config/config.yaml')
        parser.add_argument('--model',         default='bilstm',
                            choices=['bilstm', 'transformer', 'stgcn'])
        parser.add_argument('--exp',           default='exp01',
                            help='Experiment name (results saved under results/<exp>/)')
        parser.add_argument('--weighted_loss', action='store_true',
                            help='Use inverse-frequency class weights in loss')
        args = parser.parse_args()
        train(args)


    if __name__ == '__main__':
        main()
```

1.10 utils.py

[10]:
```
from IPython.display import Markdown

file_path = './src/utils.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
```

```python
# -*- coding: utf-8 -*-
"""
utils.py - Core utilities for MSL Recognition System

Handles:
  - Annotation file parsing
  - Label vocabulary construction
  - Train/val/test splitting
  - Reproducibility helpers
  - Logging setup
"""

import os
import re
import json
import random
import logging
import hashlib
from pathlib import Path
from collections import Counter
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
from sklearn.model_selection import KFold, StratifiedKFold, train_test_split

#   Logging

def get_logger(name: str, log_file: Optional[str] = None, level=logging.INFO) -> logging.Logge
    """Create a logger that writes to stdout and optionally to a file."""
    fmt = "%(asctime)s | %(levelname)-8s | %(name)s | %(message)s"
    datefmt = "%Y-%m-%d %H:%M:%S"

    logger = logging.getLogger(name)
    logger.setLevel(level)
    logger.propagate = False

    if not logger.handlers:
        sh = logging.StreamHandler()
        sh.setFormatter(logging.Formatter(fmt, datefmt))
        logger.addHandler(sh)

    if log_file:
        Path(log_file).parent.mkdir(parents=True, exist_ok=True)
        fh = logging.FileHandler(log_file, encoding="utf-8")
        fh.setFormatter(logging.Formatter(fmt, datefmt))
        logger.addHandler(fh)
```

```
    return logger


#    Reproducibility

def set_seed(seed: int = 42):
    """Set random seeds for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


#    Annotation Parsing

def parse_annotation_file(ann_path: str) -> List[Dict]:
    """
    Parse the MSL annotation file.

    Format (tab-delimited):
        normal_myanmar_text  <TAB>  msl_gloss

    Returns:
        List of dicts with keys: 'idx', 'normal_text', 'msl_gloss', 'label'
        where 'label' is the normal_text (used as class identifier).
    """
    records = []
    ann_path = Path(ann_path)

    if not ann_path.exists():
        raise FileNotFoundError(f"Annotation file not found: {ann_path}")

    with open(ann_path, 'r', encoding='utf-8') as f:
        for line_no, line in enumerate(f):
            line = line.rstrip('\n')
            if not line.strip():
                continue

            parts = line.split('\t')
            if len(parts) < 2:
                # Try space-separated as fallback
                parts = line.split('  ', 1)
            if len(parts) < 2:
                logging.warning(f"Line {line_no+1}: cannot parse → '{line}'")
                continue
```

```
            normal_text = parts[0].strip()
            msl_gloss   = parts[1].strip()

            records.append({
                'idx':         line_no,
                'normal_text': normal_text,
                'msl_gloss':   msl_gloss,
                'label':       normal_text,   # class = full Myanmar phrase
            })

    logging.info(f"Parsed {len(records)} annotation entries from {ann_path}")
    return records


def build_label_vocabulary(records: List[Dict]) -> Tuple[Dict[str, int], Dict[int, str]]:
    """
    Build label index mappings from parsed annotation records.

    Returns:
        label2idx: {'မီးချိတ်': 0, 'သဲ အိတ်': 1, ...}
        idx2label: {0: 'မီးချိတ်', 1: 'သဲ အိတ်', ...}
    """
    # Preserve order of first occurrence
    seen = {}
    for rec in records:
        lbl = rec['label']
        if lbl not in seen:
            seen[lbl] = len(seen)

    label2idx = seen
    idx2label = {v: k for k, v in label2idx.items()}

    logging.info(f"Vocabulary size: {len(label2idx)} unique classes")
    return label2idx, idx2label


def save_label_map(label2idx: Dict[str, int], idx2label: Dict[int, str], out_path: str):
    Path(out_path).parent.mkdir(parents=True, exist_ok=True)
    payload = {
        'label2idx': label2idx,
        'idx2label': {str(k): v for k, v in idx2label.items()},
        'num_classes': len(label2idx),
    }
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)
    logging.info(f"Label map saved → {out_path}")
```

```python
def load_label_map(path: str) -> Tuple[Dict[str, int], Dict[int, str]]:
    with open(path, 'r', encoding='utf-8') as f:
        payload = json.load(f)
    label2idx = payload['label2idx']
    idx2label = {int(k): v for k, v in payload['idx2label'].items()}
    return label2idx, idx2label


#    Video   Annotation Matching

def match_videos_to_annotations(
    video_dir: str,
    records: List[Dict],
    keypoint_dir: Optional[str] = None,
) -> List[Dict]:
    """
    Associate each annotation record with its keypoint (.npy) or video file.

    Matching strategy:
      1. Videos sorted by name → matched 1-to-1 with sorted annotation records
         (assumes video names are numbered or alphabetically ordered to match
          annotation file order).
      2. If a video_id_file (CSV with video_name,ann_idx) exists, use that.

    Each returned record gets extra keys:
        'video_path'   : full path to .mp4 (or None)
        'keypoint_path': full path to .npy (or None)
    """
    video_dir = Path(video_dir)
    video_extensions = {'.mp4', '.avi', '.mov', '.mkv', '.webm', '.MP4'}

    def _numeric_sort_key(path: Path):
        """
        Natural / numeric sort key so that:
          idx20-2.mp4 < idx20-10.mp4 < idx20-31.mp4 < idx20-100.mp4
        Without this, Python's default str sort gives:
          idx20-10 < idx20-100 < idx20-2 < idx20-31   ← WRONG
        which silently maps every video to the wrong annotation.
        """
        stem = path.stem   # e.g. "idx20-31"
        # Split on any run of digits and sort numerically on each piece
        parts = re.split(r'(\d+)', stem)
        return [int(p) if p.isdigit() else p.lower() for p in parts]

    # Collect all video files sorted NUMERICALLY (not alphabetically)
    all_videos = sorted(
        [p for p in video_dir.rglob('*') if p.suffix in video_extensions],
```

```
        key=_numeric_sort_key,
    )

    logging.info(f"Found {len(all_videos)} video files in {video_dir}")

    if len(all_videos) != len(records):
        logging.warning(
            f"Video count ({len(all_videos)})  annotation count ({len(records)}). "
            "Matching by position up to min length."
        )

    matched = []
    for i, rec in enumerate(records):
        entry = dict(rec)
        if i < len(all_videos):
            vp = all_videos[i]
            entry['video_path'] = str(vp)
            # Derive keypoint path
            if keypoint_dir:
                kp_path = Path(keypoint_dir) / vp.relative_to(video_dir).with_suffix('.npy')
                entry['keypoint_path'] = str(kp_path)
            else:
                entry['keypoint_path'] = None
        else:
            entry['video_path'] = None
            entry['keypoint_path'] = None

        matched.append(entry)

    return matched


#   Data Splitting

def create_splits(
    records: List[Dict],
    label2idx: Dict[str, int],
    test_ratio: float = 0.15,
    val_ratio: float  = 0.15,
    seed: int         = 42,
    output_path: Optional[str] = None,
) -> Dict[str, List[int]]:
    """
    Create train/val/test splits.

    Strategy

    For the MSL4Emergency dataset almost every class has only ONE sample,
```

```
so sklearn's stratified split is impossible (requires  2 per class).

We instead use a simple reproducible random shuffle split.
This is the correct and honest approach:
  • All 558 raw videos are split randomly (no augmentation leakage).
  • Augmentation is applied ONLY to the training portion afterwards.
  • Val/test are always evaluated on original, un-augmented keypoints.

Returns dict with keys 'train', 'val', 'test' containing record indices.
"""
labels  = [label2idx[r['label']] for r in records]
label_counts = Counter(labels)
min_count    = min(label_counts.values())

# Check whether stratified split is feasible
n_min_needed = 2   # sklearn requires at least 2 per class for stratify
can_stratify = (min_count >= n_min_needed)

if not can_stratify:
    logging.warning(
        f"Most classes have only {min_count} sample(s) - "
        "stratified splitting is not possible. "
        "Falling back to RANDOM (non-stratified) split. "
        "This is expected for the MSL4Emergency dataset where "
        "each sign has exactly one video. "
        "Augmentation will be applied to training keypoints only."
    )

indices = list(range(len(records)))

# Seed for reproducibility
rng = random.Random(seed)
shuffled = list(indices)
rng.shuffle(shuffled)

n        = len(shuffled)
n_test   = max(1, int(round(n * test_ratio)))
n_val    = max(1, int(round(n * val_ratio)))
n_train  = n - n_test - n_val

train_idx = sorted(shuffled[:n_train])
val_idx   = sorted(shuffled[n_train: n_train + n_val])
test_idx  = sorted(shuffled[n_train + n_val:])

splits = {
    'train': train_idx,
    'val':   val_idx,
    'test':  test_idx,
```

```
    }

    logging.info(
        f"Data split (random) → train: {len(train_idx)}, "
        f"val: {len(val_idx)}, test: {len(test_idx)}"
    )
    logging.info(
        f"  Classes in train: {len(set(labels[i] for i in train_idx))}, "
        f"val: {len(set(labels[i] for i in val_idx))}, "
        f"test: {len(set(labels[i] for i in test_idx))}"
    )

    if output_path:
        Path(output_path).parent.mkdir(parents=True, exist_ok=True)
        with open(output_path, 'w') as f:
            json.dump(splits, f, indent=2)
        logging.info(f"Splits saved → {output_path}")

    return splits


def create_kfold_splits(
    records: List[Dict],
    label2idx: Dict[str, int],
    n_folds: int = 5,
    seed: int    = 42,
    output_path: Optional[str] = None,
) -> List[Dict[str, List[int]]]:
    """
    Create K-Fold splits for cross-validation.

    Uses standard KFold (not StratifiedKFold) because the MSL4Emergency
    dataset has predominantly 1 sample per class, making stratification
    impossible.  The fold boundaries are reproducible via `seed`.

    Returns list of {'fold', 'train', 'val'} dicts, one per fold.
    """
    indices = np.arange(len(records))

    kf     = KFold(n_splits=n_folds, shuffle=True, random_state=seed)
    folds  = []
    for fold, (train_idx, val_idx) in enumerate(kf.split(indices)):
        folds.append({
            'fold':  fold,
            'train': train_idx.tolist(),
            'val':   val_idx.tolist(),
        })
        logging.info(f"Fold {fold}: train={len(train_idx)}, val={len(val_idx)}")
```

```
    if output_path:
        Path(output_path).parent.mkdir(parents=True, exist_ok=True)
        with open(output_path, 'w') as f:
            json.dump(folds, f, indent=2)
        logging.info(f"K-fold splits saved → {output_path}")

    return folds


def load_splits(path: str) -> Dict:
    with open(path, 'r') as f:
        return json.load(f)


#    Dataset Statistics

def print_dataset_stats(records: List[Dict], label2idx: Dict[str, int], split: str = "full"):
    """Print class distribution and dataset statistics."""
    labels        = [r['label'] for r in records]
    counter       = Counter(labels)
    # Only count classes actually present in this split
    present_cls   = len(counter)
    all_cls       = len(label2idx)
    counts        = list(counter.values())

    print(f"\n{'='*60}")
    print(f"Dataset Statistics [{split}]")
    print(f"{'='*60}")
    print(f"  Total samples    : {len(records)}")
    print(f"  Classes (present): {present_cls} / {all_cls} total")
    print(f"  Samples/class    : min={min(counts)}, "
          f"max={max(counts)}, "
          f"mean={np.mean(counts):.2f}")

    # Distribution of class sizes (important for 1-sample datasets)
    size_dist = Counter(counts)
    print(f"  Size distribution:")
    for size in sorted(size_dist.keys()):
        n = size_dist[size]
        pct = n / present_cls * 100
        print(f"    {size} sample(s) per class : {n:4d} classes  ({pct:.1f}%)")

    if split == 'full' or present_cls <= 30:
        print(f"\n  Sample listing (first 20 classes):")
        for label, cnt in counter.most_common(20):
            print(f"    {label[:40]:<40} {cnt:3d}")
    print(f"{'='*60}\n")
```

```python
#   Sequence Utilities

def pad_sequence_to_length(
    seq: np.ndarray,
    target_len: int,
    pad_value: float = 0.0,
) -> np.ndarray:
    """
    Pad or truncate a sequence to target_len.
    seq shape: (T, ...) → (target_len, ...)
    """
    T = seq.shape[0]
    if T >= target_len:
        return seq[:target_len]
    pad_shape = (target_len - T,) + seq.shape[1:]
    padding   = np.full(pad_shape, pad_value, dtype=seq.dtype)
    return np.concatenate([seq, padding], axis=0)


def compute_sequence_lengths(keypoint_dir: str, records: List[Dict]) -> Dict[int, int]:
    """Load all .npy files and record their lengths (T dimension)."""
    lengths = {}
    for rec in records:
        kp_path = rec.get('keypoint_path')
        if kp_path and Path(kp_path).exists():
            kp = np.load(kp_path, mmap_mode='r')
            lengths[rec['idx']] = kp.shape[0]
    return lengths


#   Checkpoint Helpers

def save_checkpoint(
    state: dict,
    filepath: str,
    is_best: bool = False,
    best_filepath: Optional[str] = None,
):
    """Save model checkpoint."""
    Path(filepath).parent.mkdir(parents=True, exist_ok=True)
    torch.save(state, filepath)
    if is_best and best_filepath:
        import shutil
        shutil.copyfile(filepath, best_filepath)
```

```python
def load_checkpoint(filepath: str, device: torch.device) -> dict:
    """Load model checkpoint."""
    if not Path(filepath).exists():
        raise FileNotFoundError(f"Checkpoint not found: {filepath}")
    return torch.load(filepath, map_location=device)


#   Misc

def count_parameters(model: torch.nn.Module) -> int:
    """Count trainable parameters."""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def get_device(prefer_cuda: bool = True) -> torch.device:
    """Select best available device."""
    if prefer_cuda and torch.cuda.is_available():
        device = torch.device('cuda')
        gpu = torch.cuda.get_device_name(0)
        mem = torch.cuda.get_device_properties(0).total_memory / 1024**3
        logging.info(f"Using GPU: {gpu} ({mem:.1f} GB)")
    else:
        device = torch.device('cpu')
        logging.info("Using CPU")
    return device


def compute_class_weights(
    records: List[Dict],
    label2idx: Dict[str, int],
    device: torch.device,
) -> torch.Tensor:
    """
    Compute inverse-frequency class weights for imbalanced data.

    For the MSL4Emergency dataset (1 sample/class in original data):
    after augmentation all training classes have equal counts, so
    weights will be uniform - that is correct and expected.
    Classes absent from the training split get weight=1.0 (neutral).
    """
    labels  = [label2idx[r['label']] for r in records]
    counter = Counter(labels)
    n_cls   = len(label2idx)
    total   = len(labels)

    weights = torch.ones(n_cls)   # default: neutral weight for unseen classes
    for cls_idx, cnt in counter.items():
        if cnt > 0:
```

```
            weights[cls_idx] = total / (n_cls * cnt)

    return weights.to(device)
```

1.11 For Running on Windows OS Machine

1.12 export_to_onnx.py

[11]:
```
from IPython.display import Markdown

file_path = './src/export_to_onnx.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
export_to_onnx.py  -  Export trained MSL models to ONNX format
                      (runs on Linux/HPC server side)

Bug history / fixes

  [EXPORT-A] _BiLSTMExportWrapper - lengths must be None, NOT computed from mask.

      BiLSTM.forward calls pack_padded_sequence when lengths is not None.
      During torch.onnx.export tracing the dummy mask is all-zeros, so any
      (~mask).sum() gives max_seq_len and the LSTM is traced with the full-length
      packing path baked into the ONNX file.  At inference time the baked path
      doesn't adapt to shorter sequences; more critically, if the attention pooling
      uses a lengths value that is always max_seq_len it averages over the zero-
      padded tail → wrong context vector for any video shorter than max_seq_len.

      Fix: lengths=None → LSTM runs unpacked on the full padded sequence (the else
      branch in BiLSTM.forward).  AdditiveAttention receives the mask directly and
      uses masked_fill to exclude padded positions from softmax - the mask IS a live
      ONNX input and varies correctly at inference time.

  [EXPORT-B] _TransformerExportWrapper - missing inner FFN dropout.

      PyTorch's TransformerEncoderLayer has THREE dropout objects:
        self.dropout  - inner FFN dropout, placed between activation and linear2
        self.dropout1 - post-attention residual dropout
        self.dropout2 - post-FFN residual dropout
```

```
      The FFN path in PyTorch source:
        ff = linear2(dropout(activation(linear1(x))))  ← inner dropout
        x  = x + dropout2(ff)                          ← residual dropout

      Previous wrapper omitted the inner dropout entirely:
        ff = linear2(activation(linear1(x)))   ← wrong
        x  = x + dropout2(ff)

      This is a no-op during eval() export (all dropouts are identity), so it
      does not affect current inference results.  However it is architecturally
      incorrect and would silently produce wrong outputs if the model were ever
      exported in training mode.  Added self.layer_dropout_inner ModuleList.

  [EXPORT-C] _TransformerExportWrapper - double-dropout in post-norm FFN path.
      dropout2 was applied twice in the residual add. Fixed: apply once.
      (Retained from previous revision - still correct.)

  [EXPORT-D] validate_onnx - np.bool_ for Windows ORT compatibility.

  [EXPORT-E] ST-GCN in-place copy_ during tracing → pre-bake via _STGCNExportWrapper.
  [EXPORT-F] PyTorch 2.x fused MHA has no ONNX mapping → ManualMHA.
  [EXPORT-G] GELU requires opset >= 20 → erf-based equivalent.
  [EXPORT-H] Batch size 2 dummy inputs prevent 0-D scalar-collapse.
  [EXPORT-I] 'model_type' was missing from metadata JSON → now always written.

NOTE on the SUMMARY display "0.0 MB" for JSON files:
  The metadata JSON files are ~130 bytes.  Displayed with one decimal place
  in MB (bytes / 1024^2) this rounds to 0.0.  The files are correct and
  complete - infer_onnx.py reads them fine.
"""

import argparse
import json
import shutil
import sys
from pathlib import Path

import numpy as np
import torch
import yaml

sys.path.insert(0, str(Path(__file__).parent))
from models import build_model
from utils  import get_logger, get_device, load_label_map

MODEL_CONFIGS = {
    "bilstm":       {"exp_dir": "results/exp_bilstm",       "output_name": "bilstm_msl.onnx"},
```

```
    "transformer":  {"exp_dir": "results/exp_transformer",  "output_name": "transformer_msl.onn
    "stgcn":        {"exp_dir": "results/exp_stgcn",        "output_name": "stgcn_msl.onnx"},
}

POSE_N       = 33
HAND_N       = 21
TOTAL_JOINTS = POSE_N + HAND_N * 2   # 75
FEAT_DIM     = TOTAL_JOINTS * 3      # 225


#
# BiLSTM export
#

class _BiLSTMExportWrapper(torch.nn.Module):
    """
    Wrapper for BiLSTM ONNX export.

    Fix [EXPORT-A]: lengths=None bypasses pack_padded_sequence entirely.

    BiLSTM.forward branch when lengths=None:
        lstm_out, _ = self.lstm(x)          # full padded sequence, no packing
        context = self.attention(lstm_out, mask)   # mask zeros out padded positions

    The mask (True = padded) is a live ONNX input, so attention correctly
    ignores padded frames at every inference call regardless of video length.
    """
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, keypoints: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
        return self.model(keypoints, lengths=None, mask=mask)


def export_bilstm(model, out_path: Path, max_seq_len: int, device: str):
    model.eval()
    wrapper = _BiLSTMExportWrapper(model).to(device)
    wrapper.eval()

    dummy_kp   = torch.randn(2, max_seq_len, FEAT_DIM, device=device)
    dummy_mask = torch.zeros(2, max_seq_len, dtype=torch.bool, device=device)
    print(f"  BiLSTM dummy inputs: kp={tuple(dummy_kp.shape)}, mask={tuple(dummy_mask.shape)}")

    with torch.no_grad():
        torch.onnx.export(
            wrapper, (dummy_kp, dummy_mask), str(out_path),
            opset_version=16,
```

```
            input_names=["keypoints", "mask"],
            output_names=["logits"],
            dynamic_axes={"keypoints": {0: "batch"}, "mask": {0: "batch"}, "logits": {0: "batch
            do_constant_folding=True,
            verbose=False,
        )
    print(f"  Exported → {out_path}")


#
# Transformer export
#

def _onnx_safe_gelu(x: torch.Tensor) -> torch.Tensor:
    """erf-based GELU - opset-14 compatible, avoids opset-20 requirement. [EXPORT-G]"""
    return 0.5 * x * (1.0 + torch.erf(x / 1.4142135623730951))


class ManualMHA(torch.nn.Module):
    """
    Manual Multi-Head Attention that bypasses PyTorch 2.x fused
    aten::_native_multi_head_attention (no ONNX opset-16 mapping). [EXPORT-F]
    Weights cloned from the original nn.MultiheadAttention.
    """
    def __init__(self, mha: torch.nn.MultiheadAttention):
        super().__init__()
        self.embed_dim = mha.embed_dim
        self.num_heads = mha.num_heads
        self.head_dim  = self.embed_dim // self.num_heads
        self.in_proj_weight  = torch.nn.Parameter(mha.in_proj_weight.data.clone())
        self.in_proj_bias    = torch.nn.Parameter(mha.in_proj_bias.data.clone())
        self.out_proj_weight = torch.nn.Parameter(mha.out_proj.weight.data.clone())
        self.out_proj_bias   = torch.nn.Parameter(mha.out_proj.bias.data.clone())

    def forward(self, query, key, value, key_padding_mask=None, need_weights=False):
        B, T, C = query.shape
        q_w = self.in_proj_weight[:C];         k_w = self.in_proj_weight[C:2*C];     v_w = self
        q_b = self.in_proj_bias[:C];           k_b = self.in_proj_bias[C:2*C];       v_b = self

        Q = torch.nn.functional.linear(query, q_w, q_b).reshape(B, -1, self.num_heads, self.hea
        K = torch.nn.functional.linear(key,   k_w, k_b).reshape(B, -1, self.num_heads, self.hea
        V = torch.nn.functional.linear(value, v_w, v_b).reshape(B, -1, self.num_heads, self.hea

        scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.head_dim ** 0.5)
        if key_padding_mask is not None:
            scores = scores.masked_fill(key_padding_mask.unsqueeze(1).unsqueeze(2), float('-in
        attn = torch.nn.functional.softmax(scores, dim=-1)
        out  = torch.matmul(attn, V).transpose(1, 2).reshape(B, -1, C)
```

```
        out  = torch.nn.functional.linear(out, self.out_proj_weight, self.out_proj_bias)
        return (out, attn) if need_weights else (out, None)


class _TransformerExportWrapper(torch.nn.Module):
    """
    Manually unrolls the TransformerEncoder to bypass PyTorch 2.x fused kernels.

    Fix [EXPORT-B]: Added missing inner FFN dropout (layer.dropout).
    Fix [EXPORT-C]: Residual dropout (dropout2) applied exactly once.

    PyTorch TransformerEncoderLayer has three dropout objects:
      layer.dropout  - inner FFN dropout: linear2(dropout(activation(linear1(x))))
      layer.dropout1 - post-attention residual: x + dropout1(attn_out)
      layer.dropout2 - post-FFN residual:       x + dropout2(ff_out)

    All are nn.Dropout instances, identity in eval() mode, so they do not affect
    current inference results. They are included for architectural correctness.

    The real model always uses norm_first=True (Pre-LN) so only that path runs.
    """
    def __init__(self, model):
        super().__init__()
        self.input_norm       = model.input_norm
        self.input_proj       = model.input_proj
        self.proj_norm        = model.proj_norm
        self.cls_token        = model.cls_token
        self.pos_enc          = model.pos_enc
        self.encoder_norm     = model.encoder.norm
        self.head_norm        = model.head_norm
        self.head_dropout     = model.head_dropout
        self.fc               = model.fc

        self.layer_self_attn      = torch.nn.ModuleList([ManualMHA(l.self_attn) for l in model
        self.layer_linear1        = torch.nn.ModuleList([l.linear1  for l in model.encoder.lay
        self.layer_linear2        = torch.nn.ModuleList([l.linear2  for l in model.encoder.lay
        self.layer_norm1          = torch.nn.ModuleList([l.norm1    for l in model.encoder.lay
        self.layer_norm2          = torch.nn.ModuleList([l.norm2    for l in model.encoder.lay
        self.layer_dropout1       = torch.nn.ModuleList([l.dropout1 for l in model.encoder.lay
        # Fix [EXPORT-B]: inner FFN dropout (between activation and linear2)
        self.layer_dropout_inner  = torch.nn.ModuleList([l.dropout  for l in model.encoder.lay
        self.layer_dropout2       = torch.nn.ModuleList([l.dropout2 for l in model.encoder.lay

        self.num_layers = len(model.encoder.layers)
        self.norm_first = model.encoder.layers[0].norm_first  # always True

        act = model.encoder.layers[0].activation
        self._use_gelu = not (act is torch.nn.functional.relu or isinstance(act, torch.nn.ReLU
```

```
    def forward(self, keypoints: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
        x = self.proj_norm(self.input_proj(self.input_norm(keypoints)))  # (B, T, d_model)

        B = x.shape[0]
        x = torch.cat([self.cls_token.expand(B, -1, -1), x], dim=1)     # (B, T+1, d_model)
        cls_mask     = torch.zeros(B, 1, dtype=torch.bool, device=x.device)
        padding_mask = torch.cat([cls_mask, mask], dim=1)                # (B, T+1)
        x = self.pos_enc(x)

        for i in range(self.num_layers):
            # Pre-LN (norm_first=True) - matches PyTorch TransformerEncoderLayer exactly
            nx = self.layer_norm1[i](x)
            ao, _ = self.layer_self_attn[i](nx, nx, nx,
                        key_padding_mask=padding_mask, need_weights=False)
            x = x + self.layer_dropout1[i](ao)

            nx = self.layer_norm2[i](x)
            ff = self.layer_linear1[i](nx)
            ff = _onnx_safe_gelu(ff) if self._use_gelu else torch.nn.functional.relu(ff)
            # Fix [EXPORT-B]: inner dropout (identity in eval, included for correctness)
            ff = self.layer_dropout_inner[i](ff)
            ff = self.layer_linear2[i](ff)
            # Fix [EXPORT-C]: residual dropout applied exactly once
            x  = x + self.layer_dropout2[i](ff)

        cls_out = self.encoder_norm(x)[:, 0, :]   # CLS token
        return self.fc(self.head_dropout(self.head_norm(cls_out)))


def export_transformer(model, out_path: Path, max_seq_len: int, device: str):
    model.eval()
    wrapper = _TransformerExportWrapper(model).to(device)
    wrapper.eval()

    dummy_kp   = torch.randn(2, max_seq_len, FEAT_DIM, device=device)
    dummy_mask = torch.zeros(2, max_seq_len, dtype=torch.bool, device=device)
    print(f"  Transformer dummy inputs: kp={tuple(dummy_kp.shape)}, mask={tuple(dummy_mask.sha

    with torch.no_grad():
        try:
            from torch.nn.attention import sdp_kernel
            attn_ctx = sdp_kernel(enable_flash=False, enable_mem_efficient=False, enable_math=
        except ImportError:
            import contextlib
            attn_ctx = contextlib.nullcontext()
        with attn_ctx:
            torch.onnx.export(
```

```
                wrapper, (dummy_kp, dummy_mask), str(out_path),
                opset_version=16,
                input_names=["keypoints", "mask"],
                output_names=["logits"],
                dynamic_axes={
                    "keypoints": {0: "batch", 1: "seq_len"},
                    "mask":      {0: "batch", 1: "seq_len"},
                    "logits":    {0: "batch"},
                },
                do_constant_folding=True,
                verbose=False,
            )
    print(f"  Exported → {out_path}")


#
# ST-GCN export
#

class _STGCNExportWrapper(torch.nn.Module):
    """
    Pre-bakes edge-importance weights into the adjacency buffer of each block
    so no in-place copy_() appears in the traced graph. [EXPORT-E]

    After baking, sets edge_importance_weighting=False so the STGCN forward
    uses block.gcn.A directly (the pre-scaled buffer) and the
    edge_importance ParameterList is never accessed during tracing.
    """
    def __init__(self, model):
        super().__init__()
        self.model = model
        for i, block in enumerate(model.st_gcn_blocks):
            A_base = block.gcn.A.detach().clone()
            if model.edge_importance_weighting:
                ei     = model.edge_importance[i].detach().clone()
                A_sc   = (A_base * ei).clamp(min=0)
            else:
                A_sc = A_base
            block.gcn.A.data.copy_(A_sc)
        self.model.edge_importance_weighting = False

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.model(x)


def export_stgcn(model, out_path: Path, max_seq_len: int, device: str):
    model.eval()
    wrapper = _STGCNExportWrapper(model).to(device)
```

```
    wrapper.eval()

    dummy_kp = torch.randn(2, 3, max_seq_len, TOTAL_JOINTS, device=device)
    print(f"  ST-GCN dummy input: kp={tuple(dummy_kp.shape)}")

    with torch.no_grad():
        torch.onnx.export(
            wrapper, dummy_kp, str(out_path),
            opset_version=16,
            input_names=["keypoints"],
            output_names=["logits"],
            dynamic_axes={"keypoints": {0: "batch"}, "logits": {0: "batch"}},
            do_constant_folding=True,
            verbose=False,
        )
    print(f"  Exported → {out_path}")


#
# Validation
#

def validate_onnx(out_path: Path, model_type: str, max_seq_len: int, num_classes: int):
    try:
        import onnx, onnxruntime as ort
    except ImportError:
        print("  [SKIP] onnx / onnxruntime not installed")
        return True

    print("  Validating ONNX model...")
    onnx.checker.check_model(onnx.load(str(out_path)))
    print("    ONNX graph is valid")

    sess = ort.InferenceSession(str(out_path), providers=["CPUExecutionProvider"])

    if model_type == "stgcn":
        dummy = {"keypoints": np.random.randn(1, 3, max_seq_len, TOTAL_JOINTS).astype(np.float3
    else:
        # Fix [EXPORT-D]: np.bool_ explicit - Windows ORT rejects Python built-in bool
        dummy = {
            "keypoints": np.random.randn(1, max_seq_len, FEAT_DIM).astype(np.float32),
            "mask":      np.zeros((1, max_seq_len), dtype=np.bool_),
        }

    logits = sess.run(None, dummy)[0]
    assert logits.shape == (1, num_classes), \
        f"Shape mismatch: expected (1,{num_classes}), got {logits.shape}"
    print(f"    Inference OK - shape={logits.shape}, "
```

```
          f"top-5={np.argsort(logits[0])[::-1][:5].tolist()}")
    return True


#
# Orchestrator
#

def export_model(model_type: str, cfg: dict, output_dir: Path, device: str,
                 validate: bool = True) -> bool:
    print(f"\n{'='*60}\n  Exporting  {model_type.upper()}\n{'='*60}")

    mcfg      = MODEL_CONFIGS[model_type]
    ckpt_path = Path(mcfg["exp_dir"]) / "checkpoints" / "best.pth"
    out_path  = output_dir / mcfg["output_name"]

    if not ckpt_path.exists():
        print(f"  [ERROR] Checkpoint not found: {ckpt_path}")
        return False

    ckpt        = torch.load(str(ckpt_path), map_location=device)
    num_classes = ckpt.get("num_classes", 558)
    max_seq_len = cfg["data"]["max_seq_len"]
    print(f"  Classes: {num_classes}   MaxSeqLen: {max_seq_len}")

    model = build_model(model_type, cfg, num_classes)
    model.load_state_dict(ckpt["state_dict"])
    model = model.to(device)
    model.eval()

    output_dir.mkdir(parents=True, exist_ok=True)

    if   model_type == "bilstm":      export_bilstm(model, out_path, max_seq_len, device)
    elif model_type == "transformer": export_transformer(model, out_path, max_seq_len, device)
    elif model_type == "stgcn":       export_stgcn(model, out_path, max_seq_len, device)

    if validate:
        try:
            validate_onnx(out_path, model_type, max_seq_len, num_classes)
        except Exception as e:
            print(f"  [WARNING] Validation error: {e}")

    # Fix [EXPORT-I]: metadata always includes model_type
    metadata = {
        "model_type":  model_type,
        "num_classes": num_classes,
        "max_seq_len": max_seq_len,
        "feat_dim":    FEAT_DIM,
```

```
        "num_joints":  TOTAL_JOINTS,
        "num_coords":  3,
    }
    meta_path = out_path.with_suffix(".json")
    with open(meta_path, "w") as f:
        json.dump(metadata, f, indent=2)
    print(f"  Metadata  → {meta_path}  "
          f"(~{meta_path.stat().st_size} bytes - displays as 0.0 MB, this is normal)")

    lm_src = Path(cfg["data"]["label_map_file"])
    lm_dst = output_dir / "label_map.json"
    if lm_src.exists() and not lm_dst.exists():
        shutil.copy(str(lm_src), str(lm_dst))
        print(f"  Label map → {lm_dst}")

    print(f"    Done  ({out_path.stat().st_size / 1024**2:.1f} MB)")
    return True


#
# CLI
#

def main():
    parser = argparse.ArgumentParser(description="Export MSL models to ONNX")
    parser.add_argument("--config",      default="config/config.yaml")
    parser.add_argument("--model",       choices=list(MODEL_CONFIGS.keys()))
    parser.add_argument("--all",         action="store_true")
    parser.add_argument("--output_dir",  default="onnx_models")
    parser.add_argument("--no-validate", action="store_true")
    parser.add_argument("--device",      default=None)
    args = parser.parse_args()

    if not args.model and not args.all:
        parser.error("Specify --model <name> or --all")

    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    device     = args.device or ("cuda" if torch.cuda.is_available() else "cpu")
    models     = list(MODEL_CONFIGS.keys()) if args.all else [args.model]
    output_dir = Path(args.output_dir)
    print(f"Device: {device}")

    ok = sum(export_model(m, cfg, output_dir, device, not args.no_validate) for m in models)

    print(f"\n{'='*60}\n  SUMMARY: {ok}/{len(models)} exported to '{output_dir}/'")
    for f in sorted(output_dir.glob("*")):
```

```
        size_mb = f.stat().st_size / 1024**2
        note = "  (metadata - tiny file, 0.0 MB is correct)" if f.suffix == ".json" and size_mb
        print(f"    {f.name:<35} {size_mb:6.1f} MB{note}")
    print("="*60)


if __name__ == "__main__":
    main()
```

1.13 infer_onnx.py

[12]:
```
from IPython.display import Markdown

file_path = './src/infer_onnx.py'

with open(file_path, 'r') as f:
    code = f.read()

# 'python' keyword triggers the syntax highlighter
display(Markdown(f"```python\n{code}\n```"))
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
infer_onnx.py  -  Real-time MSL recognition on Windows using ONNX Runtime

Complete bug list (this version fixes ALL of them)


  [INFER-F] model_complexity=1 for video/batch, but training used complexity=2.
      *** THIS IS THE ROOT CAUSE OF ALL WRONG PREDICTIONS ON WINDOWS ***

      MediaPipe Pose has three complexity levels (0, 1, 2).  Each uses a
      different neural network with different architecture and weights, and
      they produce MEASURABLY DIFFERENT landmark coordinate values for the
      same input frame.  The differences are small visually but large enough
      to shift the normalised feature distribution away from what the model
      was trained on.

      Evidence from the notebook comparison:
        Linux infer.py  → correct predictions, 55-93% confidence
        Windows ONNX    → wrong predictions,    2-14% confidence (10× lower)

      The 10× confidence collapse is the diagnostic signature of a feature
      distribution mismatch: the model is running correctly but its inputs
      look like nothing it has ever seen.

      How training/Linux used complexity:
```

```
    extract_keypoints.py: default --complexity 2  → all .npy files use 2
    infer.py infer_video:  _open_detectors(model_complexity=2)       ← 2
    infer.py infer_batch:  _open_detectors(model_complexity=2)       ← 2
    infer.py infer_webcam: _open_detectors(model_complexity=1)       ← 1 (speed)

  How our infer_onnx.py was using complexity:
    KeypointExtractor(model_complexity=1) everywhere                  ← WRONG

  Fix: use model_complexity=2 for video and batch modes (matching
  training and infer.py).  Keep complexity=1 for webcam (same as
  infer.py, acceptable tradeoff for real-time speed).  Expose a
  --model_complexity CLI argument so users can override if needed.

  Note: MediaPipe Hands always caps at complexity=1 (MediaPipe limit),
  so min(complexity, 1) is applied to hands in both cases.

[INFER-A] process_frame skipped frames where pose was not detected.
  Training (extract_keypoints.py / process_video) ALWAYS appends a
  keypoint array for EVERY frame.  When MediaPipe fails to detect the
  pose, extract_frame_keypoints() returns zeros for the pose block
  (and zeros for any undetected hands).  The frame is still included.

  The old code returned None when detected=False and the callers skipped
  those frames.  This caused two problems:
    1. The temporal structure of the sequence differs from training
       (missing frames change sign timing and duration).
    2. The sequence length differs from what normalization expects
       (shoulder width can become 0 in zero frames, but normalize_sequence
       handles that with the `where(dist < 1e-6, 1.0, dist)` guard).

  Fix: always return the keypoint array regardless of detection status.
  Zero rows for undetected joints are exactly what the model was trained on.

[INFER-B] normalize_sequence used 2-D shoulders and 2-D scale.
  Training (extract_keypoints.normalize_sequence) uses full 3-D midpoint
  and 3-D Euclidean shoulder width applied to ALL three coordinates.
  The old code left z unscaled (only subtracted mid_z, never divided by
  scale) and used a 2-D scale that is always smaller than the 3-D one,
  inflating x and y magnitudes.  Every prediction was wrong as a result.
  Fix: exact mirror of extract_keypoints.normalize_sequence().

[INFER-C] Webcam: buffer not cleared after each classification.
  infer.py clears the buffer after every classification so each sign is
  evaluated as a self-contained sequence, exactly as during training.
  The old code used a sliding/rolling window - fine for continuous
  recognition but poor for per-sign accuracy demos.
  Fix: clear kp_buffer after each classification, matching infer.py.
```

```
  [INFER-D] np.int64 / int dict key mismatch.
      np.argsort returns np.int64 indices.  dict.get(np.int64(k)) on an
      {int: str} dict silently returns None on some CPython/NumPy builds.
      Fix: always cast to int() before lookup via _get_label().

  [INFER-E] ONNXModel.predict double-indexed to scalar.
      [0][0] returned shape () instead of (num_classes,).
      Fix: return output[0] - shape (num_classes,).
"""

import argparse
import importlib
import json
import sys
from datetime import datetime
from pathlib import Path

import cv2
import numpy as np
import onnxruntime as ort

try:
    from PIL import Image, ImageDraw, ImageFont
    HAS_PIL = True
except ImportError:
    HAS_PIL = False

#   Constants (must match extract_keypoints.py exactly)
POSE_N          = 33
HAND_N          = 21
TOTAL_JOINTS    = POSE_N + HAND_N * 2   # 75
COORDS          = 3
FEAT_DIM        = TOTAL_JOINTS * COORDS  # 225

POSE_LEFT_SHOULDER  = 11
POSE_RIGHT_SHOULDER = 12
LEFT_HAND_START     = POSE_N            # 33
RIGHT_HAND_START    = POSE_N + HAND_N   # 54


#   MediaPipe import

def _import_mp():
    for prefix in ["mediapipe.solutions", "mediapipe.python.solutions"]:
        try:
            pose  = importlib.import_module(f"{prefix}.pose")
            hands = importlib.import_module(f"{prefix}.hands")
            draw  = importlib.import_module(f"{prefix}.drawing_utils")
```

```
            return pose, hands, draw
        except (ImportError, ModuleNotFoundError):
            continue
    try:
        import mediapipe as _mp
        return _mp.solutions.pose, _mp.solutions.hands, _mp.solutions.drawing_utils
    except AttributeError:
        pass
    raise ImportError(
        "Cannot import MediaPipe solutions.\n"
        "Try: pip uninstall mediapipe -y && pip install mediapipe==0.10.18"
    )

mp_pose, mp_hands, mp_drawing = _import_mp()


#    Myanmar text rendering

class TextRenderer:
    _FONT_CANDIDATES = [
        "C:\\Windows\\Fonts\\Padauk.ttf",
        "C:\\Windows\\Fonts\\NotoSansMyanmar-Regular.ttf",
        "C:\\Windows\\Fonts\\NotoSansMyanmar.ttf",
        "C:\\Windows\\Fonts\\myanmar3.ttf",
        "/usr/share/fonts/truetype/padauk/Padauk-Regular.ttf",
        "/usr/share/fonts/truetype/noto/NotoSansMyanmar-Regular.ttf",
        "fonts\\Padauk.ttf",
        "fonts/NotoSansMyanmar-Regular.ttf",
    ]

    def __init__(self, font_size: int = 32):
        self.font = self._load_font(font_size)
        self.can_render_myanmar = (self.font is not None)
        if not self.can_render_myanmar:
            print()
            print("                                        ")
            print("    INFO: No Myanmar shaping font found.                    ")
            print("    Displaying readable Class IDs instead of Myanmar text.  ")
            print("                                        ")
            print()

    def _load_font(self, size):
        if not HAS_PIL:
            return None
        for path in self._FONT_CANDIDATES:
            if Path(path).exists():
                try:
                    return ImageFont.truetype(path, size)
```

```
                except Exception:
                    continue
        return None

    def put_text(self, frame, text, position=(20, 20), color=(0, 255, 0)):
        if self.font and HAS_PIL:
            pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
            d   = ImageDraw.Draw(pil)
            r, g, b = color[2], color[1], color[0]
            for dx in [-1, 0, 1]:
                for dy in [-1, 0, 1]:
                    if dx or dy:
                        d.text((position[0]+dx, position[1]+dy), text,
                               font=self.font, fill=(0, 0, 0))
            d.text(position, text, font=self.font, fill=(r, g, b))
            return cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)
        cv2.putText(frame, text, position, cv2.FONT_HERSHEY_SIMPLEX,
                    0.75, color, 2, cv2.LINE_AA)
        return frame


#   Keypoint extraction

class KeypointExtractor:
    """
    Extracts (TOTAL_JOINTS, 3) keypoint arrays from BGR frames.

    BUG FIX [INFER-A]:
    process_frame now ALWAYS returns a keypoint array, never None.
    When MediaPipe fails to detect the pose, the pose block is all zeros.
    When hands are not detected, those blocks are all zeros.
    This exactly mirrors extract_keypoints.py / process_video() behaviour,
    which appends extract_frame_keypoints() output for EVERY frame without
    any None-filtering.

    Hand convention (matches extract_keypoints.py):
      MediaPipe reports handedness from the camera perspective (mirrored).
      label='Right' → signer's LEFT hand  → slot LEFT_HAND_START (33).
      label='Left'  → signer's RIGHT hand → slot RIGHT_HAND_START (54).
    """

    def __init__(self, model_complexity: int = 2):
        # Store complexity ourselves - mediapipe's Pose object does NOT expose
        # a public .model_complexity attribute in all versions (crashes on
        # mediapipe 0.10.x Windows builds with AttributeError).
        self.model_complexity = model_complexity

        self.pose  = mp_pose.Pose(
```

```
            static_image_mode=False,
            model_complexity=model_complexity,
            smooth_landmarks=True,
            enable_segmentation=False,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        )
        self.hands = mp_hands.Hands(
            static_image_mode=False,
            max_num_hands=2,
            model_complexity=min(model_complexity, 1),
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        )
        self._last_pose  = None
        self._last_hands = None

    def process_frame(self, frame: np.ndarray) -> np.ndarray:
        """
        Returns (TOTAL_JOINTS, 3) float32 array for every frame.
        Zero rows for any joint/hand that was not detected.
        Never returns None - every frame is included, same as training.
        """
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        rgb.flags.writeable = False
        p_res = self.pose.process(rgb)
        h_res = self.hands.process(rgb)
        self._last_pose  = p_res
        self._last_hands = h_res

        kp = np.zeros((TOTAL_JOINTS, COORDS), dtype=np.float32)

        # Pose - zeros if not detected (same as extract_keypoints._lm_to_array)
        if p_res.pose_landmarks:
            for i, lm in enumerate(p_res.pose_landmarks.landmark):
                kp[i] = [lm.x, lm.y, lm.z]

        # Hands - zeros for each undetected hand
        if h_res.multi_hand_landmarks and h_res.multi_handedness:
            for lm_list, handedness in zip(
                h_res.multi_hand_landmarks, h_res.multi_handedness
            ):
                label  = handedness.classification[0].label
                offset = LEFT_HAND_START if label == 'Right' else RIGHT_HAND_START
                for i, lm in enumerate(lm_list.landmark):
                    kp[offset + i] = [lm.x, lm.y, lm.z]

        # Always return the array - never None
```

```
        return kp

    def draw_landmarks(self, frame: np.ndarray):
        if self._last_pose and self._last_pose.pose_landmarks:
            mp_drawing.draw_landmarks(
                frame, self._last_pose.pose_landmarks, mp_pose.POSE_CONNECTIONS
            )
        if self._last_hands and self._last_hands.multi_hand_landmarks:
            for lm in self._last_hands.multi_hand_landmarks:
                mp_drawing.draw_landmarks(frame, lm, mp_hands.HAND_CONNECTIONS)

    def close(self):
        self.pose.close()
        self.hands.close()


#   Normalization

def normalize_sequence(seq: np.ndarray) -> np.ndarray:
    """
    BUG FIX [INFER-B]: Exact mirror of extract_keypoints.normalize_sequence().

    Translates to shoulder-midpoint origin (full 3-D) and scales by
    3-D Euclidean shoulder width.  Applied to ALL three coordinate axes.

    Training code (extract_keypoints.py):
        mid  = (l_sh + r_sh) / 2.0         shape (T, 3)
        dist = ||r_sh - l_sh||_2            shape (T, 1)  - 3-D Euclidean
        out  = (seq - mid[:, None, :]) / dist[:, None, :]

    The old code used a 2-D scale (x,y only) and did NOT divide z by scale.
    This produced inflated x/y values and wrong z values → every prediction wrong.

    Args:
        seq: (T, 75, 3) raw keypoints from KeypointExtractor.
    Returns:
        (T, 75, 3) normalised float32, matching training distribution exactly.
    """
    l_sh = seq[:, POSE_LEFT_SHOULDER,  :]   # (T, 3)
    r_sh = seq[:, POSE_RIGHT_SHOULDER, :]   # (T, 3)
    mid  = (l_sh + r_sh) / 2.0              # (T, 3)
    dist = np.linalg.norm(r_sh - l_sh, axis=1, keepdims=True)   # (T, 1)
    dist = np.where(dist < 1e-6, 1.0, dist)
    return ((seq - mid[:, None, :]) / dist[:, None, :]).astype(np.float32)


#   Preprocessor
```

```python
class Preprocessor:
    """
    Stacks raw (75, 3) keypoint frames, normalises, crops/pads, and builds
    the model-ready input dict.

    Processing order (matches infer.py exactly):
      1. Stack → (T, 75, 3)
      2. Normalize (shoulder-centred, 3-D scale, all axes)
      3. Centre-crop if T > max_seq_len
      4. Zero-pad if T < max_seq_len
      5. Build mask (True = padded position)
      6. Reshape for model type
    """

    def __init__(self, max_seq_len: int = 200):
        self.max_seq_len = max_seq_len

    def __call__(self, kp_list: list, model_type: str) -> dict:
        if not kp_list:
            return {}

        seq = np.stack(kp_list, axis=0).astype(np.float32)  # (T, 75, 3)
        seq = normalize_sequence(seq)
        T   = seq.shape[0]

        # Centre-crop
        if T > self.max_seq_len:
            start = (T - self.max_seq_len) // 2
            seq   = seq[start: start + self.max_seq_len]
            T     = self.max_seq_len

        real_len = T

        # Zero-pad
        if T < self.max_seq_len:
            pad = np.zeros((self.max_seq_len - T, TOTAL_JOINTS, COORDS), dtype=np.float32)
            seq = np.concatenate([seq, pad], axis=0)

        # Mask: True where padded
        mask = np.zeros(self.max_seq_len, dtype=np.bool_)
        mask[real_len:] = True

        if model_type == "stgcn":
            # (T, 75, 3) → (3, T, 75) → (1, 3, T, 75)
            kp_out = np.expand_dims(seq.transpose(2, 0, 1), axis=0).astype(np.float32)
            return {"keypoints": kp_out}
        else:
            # (T, 75, 3) → (T, 225) → (1, T, 225)
```

```
            kp_out = np.expand_dims(
                seq.reshape(self.max_seq_len, FEAT_DIM), axis=0
            ).astype(np.float32)
            return {"keypoints": kp_out, "mask": mask[np.newaxis]}


#    ONNX inference wrapper

class ONNXModel:
    def __init__(self, onnx_path: str, meta_path: str = None):
        available = ort.get_available_providers()
        preferred = ["CUDAExecutionProvider", "DmlExecutionProvider", "CPUExecutionProvider"]
        providers = [p for p in preferred if p in available]

        self.session = ort.InferenceSession(onnx_path, providers=providers)
        print(f"[INFO] ONNX loaded: {Path(onnx_path).name}"
              f"  (provider: {self.session.get_providers()[0]})")

        self.meta = {}
        if meta_path and Path(meta_path).exists():
            try:
                with open(meta_path, encoding="utf-8") as f:
                    self.meta = json.load(f)
            except Exception as e:
                print(f"[WARNING] Could not read metadata: {e}")
        else:
            print(f"[INFO] Metadata not found at '{meta_path}', using defaults.")

        self.model_type  = self.meta.get("model_type",  "transformer")
        self.max_seq_len = self.meta.get("max_seq_len", 200)
        self.num_classes = self.meta.get("num_classes", 558)

    def predict(self, inputs: dict) -> np.ndarray:
        """
        Run one forward pass.
        BUG FIX [INFER-E]: return 1-D (num_classes,) array, not scalar.
        output shape from session: (1, num_classes); [0] gives (num_classes,).
        """
        return self.session.run(None, inputs)[0][0]   # (num_classes,)

    @staticmethod
    def softmax(logits: np.ndarray) -> np.ndarray:
        e = np.exp(logits - logits.max())
        return e / e.sum()


#    Label map
```

```
def load_idx2label(path: str) -> dict:
    with open(path, encoding="utf-8") as f:
        data = json.load(f)
    return {int(k): v for k, v in data.get("idx2label", {}).items()}


def _get_label(idx2label: dict, class_id) -> str:
    """
    BUG FIX [INFER-D]: always cast to int() - np.int64 keys miss {int:str} dicts.
    """
    return idx2label.get(int(class_id), f"[class {int(class_id)}]")


#   Display helpers

def _build_text(class_id, label, confidence, can_myanmar,
                top_k=1, all_ids=None, probs=None, lbl_map=None):
    if top_k == 1:
        return (f"{label}  ({confidence*100:.1f}%)" if can_myanmar
                else f"Class {int(class_id)}  ({confidence*100:.1f}%)")
    parts = []
    for i, cid in enumerate(all_ids):
        lbl = _get_label(lbl_map, cid) if (can_myanmar and lbl_map) else f"Class {int(cid)}"
        parts.append(f"#{i+1} {lbl} ({probs[int(cid)]*100:.0f}%)")
    return "  |  ".join(parts)


def _build_log(class_id, label, confidence, top_k=1, all_ids=None, probs=None, lbl_map=None):
    safe = label.encode("utf-8", errors="replace").decode("utf-8")
    if top_k == 1:
        return f"  Class {int(class_id):>4d} | {safe:<40s} | {confidence*100:6.2f}%"
    lines = []
    for i, cid in enumerate(all_ids):
        lbl = _get_label(lbl_map, cid) if lbl_map else "?"
        lbl = lbl.encode("utf-8", errors="replace").decode("utf-8")
        lines.append(f"  Top-{i+1}: Class {int(cid):>4d} | {lbl:<40s} | {probs[int(cid)]*100:6
    return "\n".join(lines)


def _append_log(path, text):
    try:
        with open(path, "a", encoding="utf-8-sig") as f:
            f.write(text)
    except Exception:
        pass


#   Video processing
```

```
def run_video(model, prep, extractor, renderer, video_path,
              top_k=5, log_file=None, idx2label=None, debug=False):
    """
    Process a single video file for sign language recognition.

    Matches infer.py / process_video() behaviour exactly:
      - Every frame is processed and appended (no skipping for missing pose).
      - Normalization applied after full sequence is stacked.
      - Centre-crop or zero-pad to max_seq_len.
    """
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print(f"[ERROR] Cannot open video: {video_path}")
        return

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps_src      = cap.get(cv2.CAP_PROP_FPS) or 30.0
    print(f"[INFO] {Path(video_path).name} - {total_frames} frames @ {fps_src:.1f} fps")
    print(f"[INFO] MediaPipe pose complexity: {extractor.model_complexity}"
          f"  (must match training; training default=2)")

    kp_list = []
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        # BUG FIX [INFER-A]: process_frame always returns array (never None)
        kp_list.append(extractor.process_frame(frame))
    cap.release()

    if not kp_list:
        print("[WARNING] No frames could be read from the video.")
        return

    T_raw = len(kp_list)
    T_use = min(T_raw, model.max_seq_len)
    print(f"[INFO] {T_raw} frames extracted → using {T_use} "
          f"({'centre-cropped' if T_raw > model.max_seq_len else 'zero-padded to'}"
          f" {model.max_seq_len})")

    inputs = prep(kp_list, model.model_type)
    logits = model.predict(inputs)                    # (num_classes,)
    probs  = ONNXModel.softmax(logits)                # (num_classes,)
    top_idx = np.argsort(probs)[::-1][:top_k]

    sep = " " * 60
    lines = [f"Video  : {Path(video_path).name}",
```

```
            f"Model  : {model.model_type.upper()}", sep]
    for rank, idx in enumerate(top_idx, 1):
        lbl = _get_label(idx2label, idx) if idx2label else f"[class {int(idx)}]"
        lines.append(f"  Top-{rank}: {lbl}  ({probs[int(idx)]*100:.2f}%)")
    lines.append(sep)
    print("\n" + "\n".join(lines))

    if log_file:
        log_lines = [f"[{datetime.now().isoformat()}] {Path(video_path).name}"]
        for idx in top_idx:
            lbl = _get_label(idx2label, idx) if idx2label else "?"
            log_lines.append(_build_log(idx, lbl, probs[int(idx)],
                                        top_k=1, lbl_map=idx2label))
        _append_log(log_file, "\n".join(log_lines) + "\n\n")


#   Webcam processing

def run_webcam(model, prep, extractor, renderer, idx2label,
               cam_idx=0, top_k=1, log_file=None, classify_every=15):
    """
    Live webcam sign recognition.

    BUG FIX [INFER-A]: Every frame is always appended to the buffer -
    no None-filtering - because process_frame always returns an array.

    BUG FIX [INFER-C]: Buffer is cleared after each classification,
    matching infer.py's behaviour.  Each sign is evaluated as an
    independent self-contained sequence, exactly as during training.
    Press C to manually clear the buffer; press Q to quit.
    """
    cap = cv2.VideoCapture(cam_idx)
    if not cap.isOpened():
        print(f"[ERROR] Cannot open webcam {cam_idx}")
        return

    max_buf    = model.max_seq_len
    kp_buffer  = []
    last_text  = "Collecting frames..."
    last_color = (200, 200, 200)
    last_log   = ""
    frame_cnt  = 0
    print("[INFO] Webcam started. Sign into camera, then wait for prediction.")
    print("[INFO] Q = quit | C = clear buffer manually")

    while True:
        ret, frame = cap.read()
        if not ret:
```

```
    break

frame_cnt += 1
# BUG FIX [INFER-A]: always append, never skip
kp_buffer.append(extractor.process_frame(frame))
if len(kp_buffer) > max_buf:
    kp_buffer.pop(0)

if len(kp_buffer) >= 20 and frame_cnt % classify_every == 0:
    inputs  = prep(list(kp_buffer), model.model_type)
    logits  = model.predict(inputs)
    probs   = ONNXModel.softmax(logits)
    top_idx = np.argsort(probs)[::-1][:top_k]

    best_idx  = top_idx[0]
    best_prob = probs[int(best_idx)]
    best_lbl  = _get_label(idx2label, best_idx) if idx2label else f"Class {int(best_idx

    last_text  = _build_text(best_idx, best_lbl, best_prob,
                             renderer.can_render_myanmar,
                             top_k, top_idx, probs, idx2label)
    last_color = ((0, 200, 0) if best_prob >= 0.80 else
                  (0, 200, 200) if best_prob >= 0.50 else (0, 0, 255))

    # BUG FIX [INFER-C]: clear buffer after each classification
    # so the next sign starts fresh, matching infer.py behaviour
    kp_buffer.clear()

    if log_file and last_text != last_log:
        ts = datetime.now().strftime("%H:%M:%S")
        _append_log(log_file,
            f"[{ts}] " + _build_log(best_idx, best_lbl, best_prob,
                                   top_k=1, lbl_map=idx2label) + "\n")
        last_log = last_text

extractor.draw_landmarks(frame)
frame = renderer.put_text(frame, last_text, position=(10, 10), color=last_color)

h, w  = frame.shape[:2]
fill  = int(len(kp_buffer) / max(max_buf, 1) * w)
cv2.rectangle(frame, (0, h - 8), (fill, h), (0, 180, 255), -1)
cv2.putText(frame, f"buf {len(kp_buffer)}/{max_buf}",
            (10, h - 12), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (200, 200, 200), 1)

cv2.imshow("MSL Recognition  [Q=quit  C=clear]", frame)
key = cv2.waitKey(1) & 0xFF
if key == ord("q"):
    break
```

```
        elif key == ord("c"):
            kp_buffer.clear()
            last_text  = "Buffer cleared"
            last_color = (200, 200, 200)
            last_log   = ""

    cap.release()
    cv2.destroyAllWindows()


#    CLI

def main():
    parser = argparse.ArgumentParser(description="MSL Sign Language Inference - ONNX Runtime")
    parser.add_argument("--onnx_model",      required=True)
    parser.add_argument("--metadata",        default=None)
    parser.add_argument("--label_map",       default=None)
    grp = parser.add_mutually_exclusive_group(required=True)
    grp.add_argument("--video",  help="Path to a video file")
    grp.add_argument("--webcam", type=int, metavar="CAM_IDX")
    parser.add_argument("--top_k",          type=int, default=5)
    parser.add_argument("--classify_every", type=int, default=15,
                        help="Classify every N webcam frames (default: 15)")
    parser.add_argument("--log_file",       default=None)
    parser.add_argument("--font_size",      type=int, default=32)
    parser.add_argument("--model_complexity", type=int, default=None,
                        help="MediaPipe pose model complexity: 0, 1, or 2. "
                             "Default: 2 for video (matches training), 1 for webcam (speed). "
                             "MUST match what was used during extract_keypoints.py.")
    parser.add_argument("--debug",          action="store_true")
    args = parser.parse_args()

    model_p = Path(args.onnx_model)
    meta_p  = args.metadata or str(model_p.with_suffix(".json"))
    lmap_p  = args.label_map or str(model_p.parent / "label_map.json")

    if not Path(lmap_p).exists():
        sys.exit(f"[ERROR] label_map.json not found: {lmap_p}\n"
                 f"        Copy it from your Linux server next to the .onnx file.")

    idx2label = load_idx2label(lmap_p)
    print(f"[INFO] Loaded {len(idx2label)} labels from label map.")

    model  = ONNXModel(str(model_p), meta_p)
    prep   = Preprocessor(max_seq_len=model.max_seq_len)

    # Fix [INFER-F]: complexity must match training (=2 for video, =1 for webcam).
    # Training used extract_keypoints.py --complexity 2 (the default).
```

```
    # infer.py uses complexity=2 for video/batch, complexity=1 for webcam.
    if args.model_complexity is not None:
        video_complexity  = args.model_complexity
        webcam_complexity = args.model_complexity
    else:
        video_complexity  = 2   # matches training and infer.py infer_video()
        webcam_complexity = 1   # matches infer.py infer_webcam() - speed tradeoff

    renderer = TextRenderer(font_size=args.font_size)

    if args.log_file:
        print(f"[INFO] Logging to: {args.log_file}")

    try:
        if args.video:
            extractor = KeypointExtractor(model_complexity=video_complexity)
            try:
                run_video(model, prep, extractor, renderer,
                          args.video, args.top_k, args.log_file, idx2label, args.debug)
            finally:
                extractor.close()
        else:
            extractor = KeypointExtractor(model_complexity=webcam_complexity)
            try:
                run_webcam(model, prep, extractor, renderer, idx2label,
                           args.webcam, args.top_k, args.log_file, args.classify_every)
            finally:
                extractor.close()
    finally:
        print("[INFO] Done.")


if __name__ == "__main__":
    main()
```

[ ]: